

PRÁCTICA 1

Nombre del proyecto: Delicia Fresca

Materia:

Programación Orientada a Objetos

Grupo 1 - Equipo 3

Integrantes:

Valentina Luján Robledo – vlujan@unal.edu.co

Sebastián Estrada Villa - seestradv@unal.edu.co

Santiago Ochoa Quintero - sochoaqu@unal.edu.co

Luis David Ramírez Gonzales - luiramirezgo@unal.edu.co

Profesor:

Jaime Alberto Guzmán Luna

jaguzman@unal.edu.co

Universidad Nacional de Colombia

Sede Medellín

2024

INDICE

Descripción general de la solución	4
Identificación de los problemas y necesidades	4
Requisitos Funcionales y No Funcionales	5
Descripción del diseño estático del sistema en UML	7
Diagrama de clases y objetos del sistema	7
Descripción de las clases del sistema	9
Descripción de la implementación de características de programación orientada a objetos	16
Clase abstracta	16
Método abstracto	16
Interfaz	18
Herencia	20
Ligadura dinámica	23
Atributos de clase	25
Métodos de clase	26
Uso de Constante	27
Encapsulamiento	28
Sobrecarga de métodos	32
Sobrecarga de constructores	33
Manejo de referencias this para desambiguar y this ()	34
Caso de enumeración	37
Descripción de funcionalidades	39
Funcionalidad 1: Envío pedidos	39
Funcionalidad 2: Pago de nómina	52
Funcionalidad 3: Proveer tiendas	63

Funcionalidad 4: Devoluciones de productos	73
Funcionalidad 5: Evaluación operación	79
Manual de usuario	88
Interacción con las funcionalidades	89
Funcionalidad 1: Envío pedidos	89
Funcionalidad 2: Pago de nómina	99
Funcionalidad 3: Proveer tiendas	103
Funcionalidad 4: Devoluciones de productos	109
Funcionalidad 5: Evaluación operación	111

Descripción general de la solución

Delicia Fresca es un supermercado que ofrece productos frescos y de alta calidad, los cuales son fabricados en su propia fabrica. Todos los procesos necesarios para la operación del supermercado, desde la producción en la fábrica hasta la distribución de productos y la atención al cliente pueden ser administrados desde este programa, el cual permite llevar un control de todas las actividades del supermercado.

Identificación de los Problemas y Necesidades:

- Problemas Identificados:

- Proceso para la realización de pedidos por parte de los clientes y el envío de los mismos desde las diferentes sucursales del supermercado.
- Pago de nómina a empleados, teniendo en cuenta la cantidad de trabajo efectuado por cada empleado y el desempeño de estos en cuanto a las metas individuales propuestas por la empresa
- Gestión de inventario en fábricas y tiendas, el cual permita proveer los productos que sean necesarios de manera sencilla.
- Proceso de devoluciones de productos en base a las facturas que se generan al enviar un pedido. Estas devoluciones pueden darse por un error al hacer un envío a un cliente o por error del cliente al elegir el producto incorrecto.
- Evaluación del desempeño obtenido en las diferentes sucursales del supermercado, donde se busca conocer el rendimiento de este y así mejorar el servicio que se ofrece.

- Necesidades Específicas:

- Funcionalidad que permita a los clientes seleccionar los elementos de su pedido, para luego ser enviados a su lugar de residencia.
- Manejo de devoluciones de productos con actualización del inventario y devolución del dinero de forma inmediata.

- Sistema centralizado para manejar inventarios y pedidos, que permita tener control de los productos y la cantidad de estos que hay disponibles en cada sucursal del supermercado.
- Evaluación del trabajo realizado por los empleados y pago de las comisiones respectivas en caso de ser necesario.
- Reportes y evaluaciones operacionales que permitan conocer el rendimiento de la empresa.

Requisitos Funcionales y No Funcionales:

- Requisitos Funcionales:

A continuación, se hará una descripción de las funcionalidades diseñadas para el funcionamiento del programa.

- **Envío de pedidos:** En esta funcionalidad se gestiona el envío de pedidos desde las tiendas hasta los clientes. En esta el usuario puede seleccionar un cliente, una tienda, los productos que desea enviar y el transporte que se usará para realizar el envío según la carga de productos.
- **Pago de nómina:** En esta funcionalidad se gestiona el proceso de pago de nómina a los trabajadores de la empresa, teniendo en cuenta el trabajado realizado por cada empleado y las comisiones por el cumplimiento de las metas individuales de cada empleado. Estas metas son diferentes para cada tipo de empleado (Vendedores, Transportadores y Operarios).
- **Proveer tiendas:** En esta funcionalidad se gestiona el suministro de productos desde la fábrica hasta las diferentes tiendas, teniendo en cuenta la necesidad de estos según el inventario y la capacidad de cada tienda. Esta funcionalidad permite seleccionar tiendas, productos y tipo de transporte según la carga.
- **Proceso de devoluciones:** En esta funcionalidad se gestionan las devoluciones de los productos registrados en las facturas, para ello se deben seleccionar la factura y el producto para realizar la devolución.
- **Evaluación de operaciones:** En esta funcionalidad se gestiona la evaluación de las operaciones del supermercado, incluyendo datos como las ganancias por día,

las ganancias totales, el promedio por día, el porcentaje de aumento y las modas estadísticas. Estos datos permitirán analizar el rendimiento del supermercado y así lograr mejorar las ventas.

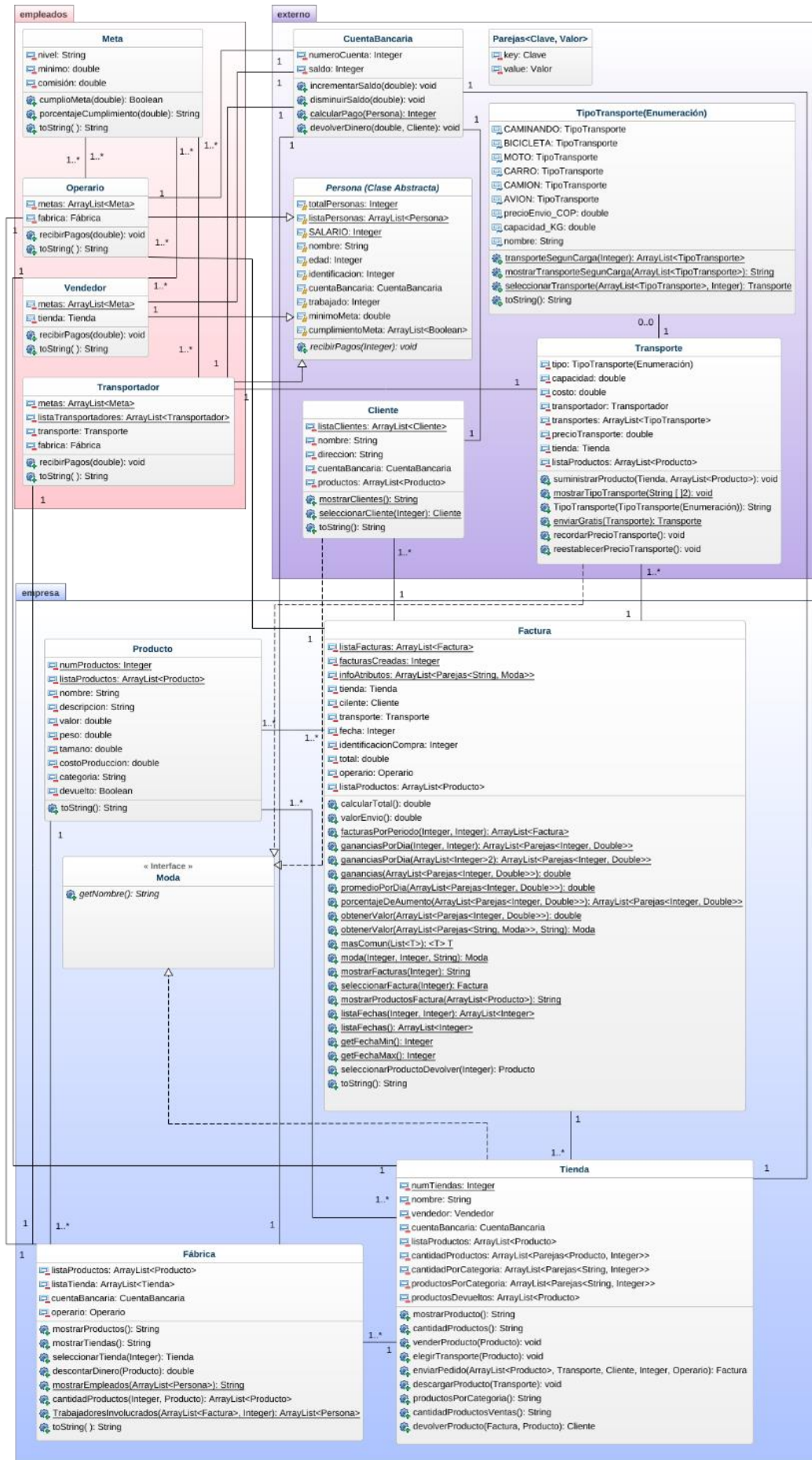
- **Requisitos No Funcionales:**

- **Rendimiento:** El sistema debe ser rápido y eficiente, permitiendo al usuario realizar el control del supermercado de manera ágil y sencilla.
- **Seguridad:** Protección de datos financieros y personales mediante el encapsulamiento y la visualización en pantalla de únicamente los datos necesarios.
- **Usabilidad:** Se implementa una interfaz intuitiva y fácil de usar, que permite utilizar el programa a pesar de no tener ningún tipo de conocimiento relacionado con programación orientada a objetos.
- **Escalabilidad:** Capacidad de manejar un aumento en la carga de trabajo y facilitar la realización de los diferentes procesos necesarios para el funcionamiento de un supermercado.

Descripción del diseño estático del sistema en UML

Diagrama de clases y objetos del sistema

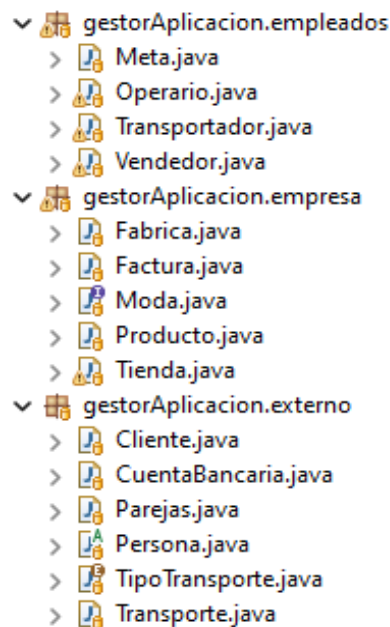
https://app.genmymodel.com/api/projects/_lgoWMNQ2Ee6YcvViuOreEw/diagrams/_lgsnoNQ2Ee6YcvViuOreEw/svg



Descripción de las clases del sistema

Este proyecto está compuesto por 13 clases, 1 interfaz y 1 enumerado. Estas clases se encuentran dentro del paquete gestorAplicacion y a su vez se encuentran divididas en otros tres paquetes: empleados, empresa y externo.

En la imagen que se encuentra a continuación, se puede apreciar mejor la división de los paquetes utilizada para la realización del proyecto.



A continuación, se presenta una breve descripción de las clases y sus relaciones en el sistema, haciendo claridad de la distribución de estas en los diferentes paquetes.

- Paquete **empleados**:

En este paquete se encuentran las clases relacionadas con la mano de obra de la empresa, incluyendo a todos los empleados y las metas que se generan para cada uno de ellos.

- **Meta:**

Esta clase representa los objetivos y metas de desempeño y/o rendimiento para los empleados. Estas metas pueden llevar a una bonificación en el pago del empleado en

caso de que se cumplan. Cada **Meta** tiene atributos como *nivel*, *minimo* y *comision*, y métodos como *cumplioMeta()*, *porcentajeCumplimiento()*, entre otros.

Relaciones:

- ❖ Está asociada con **Operario**, **Transportador** y **Vendedor**, cada uno de estos empleados puede tener múltiples metas (1..*).

- **Operario:**

Esta clase representa a un operario en la fábrica, que es el trabajador encargado de supervisar y controlar la producción de los diferentes productos en la fábrica. Cada **Operario** tiene atributos como *metas* y *fabrica*, y métodos como *recibirPagos()* y *toString()*.

Relaciones:

- ❖ Hereda de la clase abstracta **Persona**.
- ❖ Cada **Operario** tiene una única **Fábrica**.
- ❖ Cada **Operario** puede tener múltiples **Metas**.
- ❖ Cada **Operario** tiene una única **CuentaBancaria**.

- **Transportador:**

Esta clase representa a un trabajador encargado de transportar productos en determinados transportes, tanto desde la fábrica hasta las tiendas como desde las tiendas hasta los clientes. Cada **Transportador** tiene atributos como *metas* y *fabrica*, y métodos como *recibirPagos()* y *toString()*.

Relaciones:

- ❖ Hereda de la clase abstracta **Persona**.
- ❖ Cada **Transportador** tiene un único **Transporte**.
- ❖ Cada **Transportador** puede tener múltiples **Metas**.
- ❖ Cada **Transportador** tiene una única **CuentaBancaria**.
- ❖ Cada **Transportador** tiene una única **Fábrica**.

- **Vendedor:**

Esta clase representa a un trabajador encargado de ser el vendedor en alguna de las tiendas disponibles. Cada **Vendedor** tiene atributos como *metas* y *tienda*, y métodos como *recibirPagos()* y *toString()*.

Relaciones:

- ❖ Hereda de la clase abstracta **Persona**.
- ❖ Cada **Vendedor** tiene una única **Tienda**.
- ❖ Cada **Vendedor** puede tener múltiples **Metas**.
- ❖ Cada **Vendedor** tiene una única **CuentaBancaria**.

- Paquete **empresa**:

En este paquete se encuentran las clases que son exclusivamente del supermercado y que resultan ser la base de la producción y almacenamiento de los elementos que son necesarios, por lo que resultan fundamentales y de amplio uso a lo largo del programa.

• **Fabrica:**

Esta clase es la encargada de gestionar la producción y el inventario de la fábrica, aquí es donde se da la fabricación de los productos y su posterior distribución en las diferentes tiendas. En el programa existe una única fabrica desde la cual se suministran los productos a todas las tiendas. La **Fabrica** tiene atributos como *listaProductos* y *listaTienda*, y métodos como *mostrarProductos()* y *toString()*, entre otros.

Relaciones:

- ❖ Cada **Fabrica** puede tener múltiples **Tiendas**.
- ❖ Cada **Fabrica** puede tener múltiples **Productos**.
- ❖ Cada **Fabrica** tiene una única **CuentaBancaria**.
- ❖ Cada **Fabrica** tiene un único **Operario**.

• **Factura:**

Esta clase es la encargada de registrar la información importante acerca de las ventas y gestionar las devoluciones de productos. Cada **Factura** tiene atributos como *listaFacturas*, *facturasCreadas*, *infoAtributos*, *tienda*, *cliente*, *transporte*, entre otros. Tiene métodos como *calcularTotal()*, *facturasPorPeriodo()*, *gananciasPorDia()*, entre otros.

Relaciones:

- ❖ Cada **Factura** puede tener múltiples **Productos**.
- ❖ Cada **Factura** tiene un único **Cliente**.

- ❖ Cada **Factura** tiene un único **Operario**.
- ❖ Cada **Factura** tiene una única **Tienda**.
- ❖ Cada **Factura** tiene un único **Transporte**.
- ❖ Cada **Factura** puede tener múltiples **Parejas**.

- **Moda (Interfaz)**

Esta clase representa una interfaz que permite seleccionar objetos para poder encontrar la moda en la funcionalidad de evaluación operación. Esta interfaz tiene un único método llamado *getNombre()*.

Relaciones:

- ❖ **Tienda** implementa la interfaz **Moda**.
- ❖ **Transporte** implementa la interfaz **Moda**.
- ❖ **Cliente** implementa la interfaz **Moda**.

- **Producto:**

Esta clase representa los productos disponibles en la tienda y fábrica. En esta se encuentra toda la información necesaria referente a estos productos. Cada **Producto** tiene atributos como *nombre*, *descripcion*, *valor*, *peso*, *tamaño*, *costoProduccion*, *devuelto*, entre otros.

Relaciones:

- ❖ Cada **Producto** puede estar en múltiples **Tiendas**.
- ❖ Cada **Producto** puede estar en múltiples **Fabricas**.
- ❖ Cada **Producto** puede estar en múltiples **Facturas**.

- **Tienda:**

Esta clase representa la tienda donde se gestiona el inventario y se realizan las ventas de los productos. Cada **Tienda** tiene atributos como *nombre*, *vendedor*, *cuentaBancaria*, entre otros. Tiene métodos como *mostrarProductos()*, *seleccionarTienda()*, *enviarProducto()*, *descargarProducto()*, entre otros.

Relaciones:

- ❖ **Tienda** implementa la interfaz **Moda**.
- ❖ Cada **Tienda** puede tener múltiples **Productos**.
- ❖ Cada **Tienda** tiene una única **CuentaBancaria**.

- ❖ Cada **Tienda** tiene un único **Vendedor**.
- ❖ Cada **Tienda** puede tener múltiples **Parejas**.

- Paquete **externo**:

En este paquete se encuentran las clases que no son de uso exclusivo del supermercado, pero que a pesar de esto resultan importantes para la ejecución del programa y sus funcionalidades.

- **Cliente:**

Esta clase representa a los clientes que compran productos en las diferentes tiendas. Cada **Cliente** tiene atributos específicos como *direccion*, *nombre*, entre otros. Tiene métodos como *mostrarClientes()*, *seleccionarCliente()*, entre otros.

Relaciones:

- ❖ **Cliente** implementa la interfaz **Moda**
- ❖ Cada **Cliente** puede tener múltiples **Productos**.
- ❖ Cada **Cliente** tiene una única **CuentaBancaria**.

- **CuentaBancaria:**

Esta clase representa una cuenta bancaria en la que se gestionan las transacciones financieras, incluyendo acciones como realizar los pagos y devoluciones de dinero, tanto para las cuentas bancarias de los clientes como para las cuentas bancarias de los empleados y la empresa.

Cada **CuentaBancaria** tiene atributos como *numeroCuenta*, *saldo*, y métodos como *incrementarSaldo()*, *disminuirSaldo()*, *calcularPago()*, y *devolverDinero()*.

Relaciones:

- ❖ Cada **CuentaBancaria** puede pertenecer a un único *Cliente*, *Vendedor*, *Operario*, *Transportador*, *Fabrica* o *Tienda*.
- ❖ Para efectos del programa, se creó una instancia de **CuentaBancaria** para la empresa, la cual comparten las instancias de *Fabrica* y *Tienda*.

- **Parejas<K, V>:**

Esta clase es una clase genérica creada para manejar pares de valores clave (Clave) y valor (Valor), sin la necesidad de usar diccionarios.

Relaciones:

- ❖ **Parejas** es utilizada en **Tienda** para manejar pares de valores.
- ❖ **Parejas** es utilizada en **Factura** para manejar pares de valores.

- **Persona (Clase abstracta):**

Esta clase representa la clase base para Operario, Transportador y Vendedor. Esta es una clase abstracta que proporciona atributos y métodos para la creación de los diferentes empleados de la empresa.

La clase **Persona** tiene atributos y métodos comunes a las personas, como nombre, *edad*, *identificacion*, *cuentaBancaria*, entre otros. También, tiene un método abstracto (*recibirPagos()*) que debe ser implementado por sus subclases.

Relaciones:

- ❖ **Vendedor** hereda de **Persona**.
- ❖ **Operario** hereda de **Persona**.
- ❖ **Transportador** hereda de **Persona**.

- **TipoTransporte:**

Esta clase representa una enumeración de unos valores constantes que representan los tipos de transporte disponibles para realizar el transporte de los productos. Esta define diferentes tipos de transporte como *bicicleta*, *moto*, *carro*, *avión*, entre otros; los cuales poseen atributos como *precioEnvio_COP*, *capacidad_KG* y *nombre*, y métodos como *transporteSegunCarga()*, *seleccionarTransporte()*, entre otros.

Relaciones:

- ❖ Los valores de **TipoTransporte** son utilizados en **Transporte**.

- **Transporte:**

Esta clase gestiona el transporte de productos, representando los diferentes medios de transporte que se pueden utilizar para realizar el transporte de los productos.

Cada **Transporte** tiene atributos como *tipo*, *transportador*, *tienda*, entre otros. Tiene métodos como *mostrarTipoTransporte()*, *tipoTransporte()*, *enviarGratis()*, entre otros.

Relaciones:

- ❖ **Transporte** implementa la interfaz **Moda**.
- ❖ Cada **Transporte** tiene un único **TipoTransporte**.
- ❖ Cada **Transporte** tiene un único **Transportador**.
- ❖ Cada **Transporte** puede tener múltiples **Productos**.

Descripción de la implementación de características de programación orientada a objetos

- Clase abstracta:

Una clase abstracta es una clase que no puede ser instanciada directamente y está destinada a ser heredada por otras clases.

- **Uso:** Se usa específicamente como una clase base que modela el comportamiento y establece métodos que deben ser implementados por las clases derivadas, que en este caso son **Operario**, **Transportador** y **Vendedor**. Esto asegura que todas las subclases compartan un contrato común y promueve la reutilización del código.
- **Beneficios y mejoras:** Facilita la definición de una estructura común y estandariza el comportamiento de las subclases. También, mejora la consistencia y reduce la duplicación de código.
- **Ubicación:** Este comportamiento se puede observar en la clase **Persona**, presente en el archivo **Persona.java** en el paquete **gestorAplicacion.externo**

```
29 public abstract class Persona implements Serializable{
30
31     // ATRIBUTOS -----
32
33     // De clase
34     protected static final long serialVersionUID = 1L; // Versión del serializado asociada a esta clase
35     protected static int totalPersonas;
36     protected static ArrayList<Persona> listaPersonas = new ArrayList<Persona>();
37     protected static final int SALARIO = 50000;
38
39     // De instancia
40     protected String nombre;
41     protected int edad;
42     protected int identificacion;
43     protected CuentaBancaria cuentaBancaria;
44     protected int trabajado;
45     protected double minimoMeta;
46     protected ArrayList<Boolean> cumplimientoMetas = new ArrayList<Boolean>();
47 }
```

- Método abstracto

Los métodos abstractos son aquellos que no tienen implementación en la clase abstracta y deben ser implementados por las clases derivadas.

- **Uso:** Las subclases de **Persona** (**Operario**, **Transportador**, **Vendedor**) implementan el método *recibirPagos()*, proporcionando la lógica específica para cada tipo de empleado.
- **Beneficios y mejoras:** Permite a las subclases definir su comportamiento específico mientras garantiza que se sigan las directrices establecidas por la clase abstracta, con esto se mejora la flexibilidad y la usabilidad del sistema.
- **Ubicación:** La definición del método abstracto se puede observar en la clase **Persona** en el método *recibirPagos()* que se encuentra en la **línea 98**, presente en el archivo **Persona.java** en el paquete **gestorAplicacion.externo**

```

89  /**
90   * Método abstracto para realizar pagos.
91   *
92   * @param pago Monto del pago a realizar
93   *
94   * Funcionalidades en las que se usa: Su implementación en las clases 'Operario',
95   * 'Transportador' y 'Vendedor' es usada en la funcionalidad Pago de nomina
96   */
97
98   public abstract void recibirPagos(double pago); // Método abstracto
99

```

Adicionalmente, se puede visualizar la implementación individual en cada una de las clases que heredan de **Persona**, que se encuentran en el paquete **gestorAplicacion.empleados**:

- En la clase **Operario** en la **línea 109**

```

108  @Override
109  public void recibirPagos(double pago) {
110
111      fabrica.getCuentaBancaria().disminuirSaldo(pago);
112      this.getCuentaBancaria().incrementarSaldo(pago);
113  }

```

- En la clase **Transportador** en la **línea 95**

```

94  @Override
95  public void recibirPagos(double pago) {
96
97      fabrica.getCuentaBancaria().disminuirSaldo(pago);
98      super.getCuentaBancaria().incrementarSaldo(pago);
99  }

```

- En la clase **Vendedor** en la **línea 109**

```

108 @Override
109 public void recibirPagos(double pago){
110
111     tienda.getCuentaBancaria().disminuirSaldo(pago);
112     super.getCuentaBancaria().incrementarSaldo(pago);
113 }
114

```

- Interfaz

Una interfaz define un contrato que otras clases deben seguir. Las clases que implementan la interfaz deben proporcionar una implementación para todos los métodos definidos en la interfaz.

- **Uso:** La interfaz **Moda** permite seleccionar objetos para poder encontrar la moda en la funcionalidad de **evaluación operación**. Por esto, se implementa en las clases **Tienda**, **Transporte** y **Cliente**, que dentro de la funcionalidad de evaluación operación se evalúan para saber cuál instancia de estas clases es la más usada.
- **Beneficios y mejoras:** Facilita la implementación de comportamientos comunes en clases no relacionadas jerárquicamente. Promueve la flexibilidad y permite simular la herencia múltiple en el diseño del sistema.
- **Ubicación:** Este comportamiento se puede observar en la interfaz **Moda**, presente en el archivo **Moda.java** en el paquete **gestorAplicacion.empresa**

```

19 public interface Moda {
20
21     // MÉTODOS -----
22
23
24     /**
25      * Obtiene el nombre del elemento.
26      *
27      * @return el nombre del elemento como un String.
28      *
29      * Funcionalidades en las que se usa: Evaluacion operacion
30      */
31
32     public String getNombre(); // El compilador entiende que es abstracto por la falta de llaves
33
34 }

```

Esta interfaz es implementada por las clases:

- **Tienda** (En el paquete **gestorAplicacion.empresa**)

```

43 public class Tienda implements Moda, Serializable{

```

- **Cliente** (En el paquete **gestorAplicacion.externo**)

```

32 public class Cliente implements Moda, Serializable {

```

- **Transporte** (En el paquete **gestorAplicacion. externo**)

```

39 public class Transporte implements Moda, Serializable{

```

En estas clases podemos ver las diferentes implementaciones que se le dan al método `getNombre()`:

- En la clase **Tienda** en la **línea 378**

```

378 public String getNombre() { //De la interfaz Moda
379
380     return nombre;
381 }

```

- En la clase **Cliente** en la **línea 139**

```

139 public String getNombre() { // De la interfaz Moda
140
141     return nombre;
142 }

```

- En la clase **Transporte** en la **línea 286**

```

286 public String getNombre(){ //De la interfaz Moda
287
288     return tipo.getNombre();
289 }

```

- Herencia

La herencia permite que una clase derive de otra clase, heredando sus métodos y atributos. Esto promueve la reutilización de código y la creación de una jerarquía de clases.

- **Uso:** Las subclases **Operario**, **Transportador**, y **Vendedor** heredan los atributos y métodos de **Persona**, como *nombre*, *edad*, *identificacion*, y *cuentaBancaria*, permitiendo la reutilización de código y la implementación de métodos específicos como *recibirPagos()*.
- **Beneficios y mejoras:** Reduce la duplicación de código, promueve la consistencia entre elementos del sistema y permite la extensión del comportamiento de las clases base.
- **Ubicación:**

Clase base: Persona (abstracta) que se encuentra en el paquete **gestorAplicacion.externo**

```

29 public abstract class Persona implements Serializable{
30
31     // ATRIBUTOS -----
32
33     // De clase
34     protected static final long serialVersionUID = 1L; // Versión del serializado asociada a esta clase
35     protected static int totalPersonas;
36     protected static ArrayList<Persona> listaPersonas = new ArrayList<Persona>();
37     protected static final int SALARIO = 50000;
38
39     // De instancia
40     protected String nombre;
41     protected int edad;
42     protected int identificacion;
43     protected CuentaBancaria cuentaBancaria;
44     protected int trabajado;
45     protected double minimoMeta;
46     protected ArrayList<Boolean> cumplimientoMetas = new ArrayList<Boolean>();
47

```

Subclases: Operario, Transportador y Vendedor. Estas tres clases se encuentran en el paquete `gestorAplicacion.empleados`

- **Operario**

```

30 public class Operario extends Persona {

```

- **Transportador**

```

31 public class Transportador extends Persona{

```

- **Vendedor**

```

30 public class Vendedor extends Persona {

```

Si observamos a detalle estas subclases, podemos observar que se heredan todos los atributos de **Persona**, permitiendo que en estas clases declaren únicamente sus atributos específicos. Para mostrarlo, veremos la clase **Operario**:

```

30 public class Operario extends Persona {
31
32     // ATRIBUTOS-----
33
34     // De clase
35 private static ArrayList<Meta> meta = new ArrayList<Meta>(List.of(
36     new Meta("Facil",4,5000),
37     new Meta("Dificil",9,15000)); //Crear una lista inmutable con las metas de un objeto de tipo operario
38
39     //De instancia
40 private Fabrica fabrica;
41

```

Ahora, al observar el constructor principal de esta misma clase, podemos ver que se llama al constructor de la clase padre en la **línea 61** para asignar los atributos que han sido heredados:

```

59 public Operario(String nombre, int edad, int identificacion, CuentaBancaria cuentaBancaria, Fabrica fabrica) {
60
61     super(nombre, edad, identificacion, cuentaBancaria);
62     this.fabrica = fabrica;
63 }
--

```

De la misma manera, se puede observar en **Transportador** y **Vendedor**:

- **Transportador**

Atributos:

```

31 public class Transportador extends Persona{
32
33     // ATRIBUTOS-----
34
35     // De clase
36 private static ArrayList<Meta> metas = new ArrayList<Meta>(List.of(
37     new Meta("Facil",5,10000),
38     new Meta("Dificil",10,20000)); //Crear una lista inmutable
39 private static ArrayList<Transportador> listaTransportadores = new ArrayList<Transportador>();
40
41     // De instancia
42 private Transporte transporte;
43 private Fabrica fabrica;

```

Constructor:

```

62 public Transportador(String nombre, int edad, int identificacion, CuentaBancaria cuentaBancaria,Transporte transporte) {
63
64     super(nombre, edad, identificacion, cuentaBancaria);
65     this.transporte = transporte;
66     listaTransportadores.add(this);
67 }

```

- **Vendedor**

Atributos:

```

30 public class Vendedor extends Persona {
31
32     // ATRIBUTOS-----
33
34     // De clase
35     private static ArrayList<Meta> metas = new ArrayList<Meta>(List.of(
36         new Meta("Facil",6,12000),
37         new Meta("Dificil",12,25000))); //Crear una lista inmutable
38
39     // De instancia
40     private Tienda tienda;
41

```

Constructor:

```

59 public Vendedor(String nombre, int edad, int identificacion, CuentaBancaria cuentaBancaria,Tienda tienda) {
60
61     super(nombre, edad, identificacion, cuentaBancaria);
62     this.tienda=tienda;
63 }

```

- Ligadura dinámica

La ligadura dinámica permite que el método que se debe ejecutar se determine en tiempo de ejecución, basado en el objeto real que llama al método.

- **Uso:**

Uso 1: El método *mostrarEmpleados()* de la clase fabrica, donde se usa un apuntador de la clase **Persona** para una lista que contiene objetos de las subclases **Vendedor**, **Operario** y **Transportador**, y el compilador debe decidir el método *toString()* de cuál de estas clases utiliza teniendo en cuenta el objeto.

Uso 2: En los objetos de clases que implementan la interfaz **Moda**, se debe hacer la implementación del método *getNombre()*. Luego, en la funcionalidad **evaluación operación** se utiliza el método *moda()*, el cual devuelve un objeto de tipo **Moda** al que se le debe aplicar el método *getNombre()*, y aquí el compilador debe decidir el método de cuál de las clases que implementan **Moda** utiliza (**Tienda**, **Transporte**, **Cliente**)

- **Beneficios y mejoras:** Permite la flexibilidad y extensibilidad del código al permitir que los métodos apropiados se llamen según el tipo real del objeto. También, resulta útil para reducir la cantidad de código, pudiéndose crear métodos que reciban objetos que comparten un tipo, pero tienen una implementación específica del método o atributo que se busca manipular o hacer uso (Esto sirve principalmente para casos en que se tenga herencia o se haga uso de interfaces).

- **Ubicación:**

1. En el paquete **gestorAplicacion.empresa** en la clase **Fabrica** en el método **mostrarEmpleados** en la **línea 206**, viendo la aplicación de ligadura dinámica específicamente en la **línea 212**.

```
206 public static String mostrarEmpleados(ArrayList<Persona> trabajadores) {
207
208     String mensaje = "";
209     int numero = 1;
210
211     for (Persona i: trabajadores) {
212         mensaje += "\n" + "Trabajador "+ numero + i.toString(); //Uso de ligadura dinámica
213         numero++;
214     }
215
216     return mensaje;
217 }
```

2. En el paquete **uiMain** en la clase **EvaluacionOperacion** en el método *modas()* en la **línea 228, 236 y 244**


```

222 // Manejo de opciones
223 switch(opcion){
224
225     case 1:
226
227         // Uso de ligadura dinamica
228         System.out.println("La tienda más usada ha sido " + Factura.moda(fechal, fecha2, "tienda").getNombre());
229
230         break;
231
232     case 2:
233
234         // Uso de ligadura dinamica
235         System.out.println("El transporte más usado ha sido " + Factura.moda(fechal, fecha2, "transporte").getNombre());
236
237         break;
238
239     case 3:
240
241         // Uso de ligadura dinamica
242         System.out.println("El cliente al que más se le ha vendido ha sido " + Factura.moda(fechal, fecha2, "cliente").getNombre());
243
244         break;
245
246
247

```

- Atributos de clase

Los atributos de clase pertenecen a la clase y no a instancias individuales de la clase. Son útiles para almacenar datos compartidos por todas las instancias de la clase.

- **Uso:** Los atributos de clase se utilizan en varias de las clases, para declarar atributos que son comunes para todas las instancias de esa clase
- **Beneficios y mejoras:** Facilita la gestión de datos comunes a todas las instancias de una clase, promoviendo la coherencia y reduciendo la duplicación de datos y declaración de valores.
- **Ubicación:** Algunas clases que tienen atributos de clase son las clases **Operario** (línea 34), **Transportador** (línea 35) y **Vendedor** (línea 34) del paquete **gestorAplicacion.empleados**, donde el atributo de clase se usa para definir las metas de cada tipo de trabajador. Adicionalmente, en la **línea 39** de la clase **Transportador** se crea *ListaTransportadores* como atributo de clase.

En la clase **Operario** se define un atributo llamado *meta* que es común para todos los operarios.

```

30 public class Operario extends Persona {
31
32     // ATRIBUTOS-----
33
34     // De clase
35 private static ArrayList<Meta> meta = new ArrayList<Meta>(List.of(
36     new Meta("Facil",4,5000),
37     new Meta("Dificil",9,15000))); //Crear una lista inmutable con las metas de un objeto de tipo operario
38

```

En la clase **Transportador** se define un atributo llamado *meta* que es común para todos los transportadores y un atributo llamado *listaTransportadores* que sirve para almacenar todos los objetos de tipo **Transportador** que han sido creados.

```

31 public class Transportador extends Persona{
32
33     // ATRIBUTOS-----
34
35     // De clase
36 private static ArrayList<Meta> metas = new ArrayList<Meta>(List.of(
37     new Meta("Facil",5,10000),
38     new Meta("Dificil",10,20000))); //Crear una lista inmutable
39 private static ArrayList<Transportador> listaTransportadores = new ArrayList<Transportador>();

```

En la clase **Vendedor** se define un atributo llamado *meta* que es común para todos los vendedores.

```

30 public class Vendedor extends Persona {
31
32     // ATRIBUTOS-----
33
34     // De clase
35 private static ArrayList<Meta> metas = new ArrayList<Meta>(List.of(
36     new Meta("Facil",6,12000),
37     new Meta("Dificil",12,25000))); //Crear una lista inmutable
38
39     // De instancia
40 private Tienda tienda;

```

- Métodos de clase

Los métodos de clase pertenecen a la clase y no a instancias individuales de la clase. Son útiles para manipular datos compartidos por todas las instancias de la clase.

- **Uso:** Los métodos de clase se utilizan en varias de las clases, para trabajar atributos o datos que son comunes para todas las instancias de esa clase.
- **Beneficios y mejoras:** Facilita la manipulación de datos comunes a todas las instancias de una clase.
- **Ubicación:** Una de las clases que utiliza métodos de clase es la clase **Cliente** en el paquete **gestorAplicacion.externo**, donde los métodos *mostrarClientes()* en la **línea 87** y *seleccionarCliente()* en la **línea 112** son métodos de clase. Ambos métodos manipulan o hacen uso del atributo de clase *listaClientes* de la clase **Cliente**.

```

87 public static String mostrarClientes() {
88
89     String mensaje = "";
90     int numero = 1;
91
92     for(Cliente cliente:listaClientes) {
93         if (!mensaje.contains(cliente.getNombre())) {
94             mensaje += numero + ". " + cliente + "\n";
95             numero++;
96         }
97     }
98
99     return mensaje;
100 }

112 public static Cliente seleccionarCliente(int indice) {
113
114     if (indice-1 >= 0 && indice <= listaClientes.size()) {
115         return listaClientes.get(indice);
116     } else {
117         return null;
118     }
119 }

```

- Uso de Constante

Las constantes son valores que no cambian y son definidos usando *final*. Se utilizan para definir valores fijos que no deben ser modificados.

- **Uso:** Se define el atributo *SALARIO*, el cual se mantiene como valor fijo para cada una de las clases que heredan de **Persona**
- **Beneficios y mejoras:** Asegura que valores específicos permanezcan constantes durante la ejecución del programa, mejorando la estabilidad y permitiendo que se cumpla con los requerimientos del sistema.
- **Ubicación:** Atributo **salario** en la clase **Persona** en la **línea 37** en el paquete **gestorAplicacion.externo**

```
37 | protected static final int SALARIO = 50000;
```

- Encapsulamiento

El encapsulamiento es el concepto de ocultar los datos internos de una clase y permitir el acceso solo a través de métodos públicos. En este proyecto se utilizan los modificadores de acceso *private*, *protected*, *package* y *public*.

- **Uso:** Protege los datos de la clase y expone solo lo necesario mediante métodos públicos, mejorando la seguridad y el mantenimiento del código. Esto permite controlar cómo se accede y modifica la información, asegurando que los datos siempre se mantengan en un estado válido.
- **Beneficios y mejoras:** Los datos internos están protegidos contra accesos no autorizados, reduciendo el riesgo de manipulación indebida. También, facilita el mantenimiento del código, ya que los detalles de implementación están ocultos y solo se interactúa con métodos públicos.
- **Ubicación:** En la mayoría de las clases se utilizan atributos privados y métodos públicos. En la clase **Persona** se utilizan atributos con el modificador *protected* para facilitar la herencia sin dejar de lado la privacidad de estos atributos.

1. Private:

Los atributos privados protegen los datos internos de las clases, permitiendo su acceso únicamente a través de métodos específicos (getters y setters). Esto asegura que los

datos sean manipulados de manera controlada. En este sistema, la mayoría de clases cuentan con atributos privados.

Un ejemplo de esto es la clase **CuentaBancaria**, donde en la **línea 42 y 43** se declaran los atributos como *privados*, y luego a partir de la **línea 145** se definen los *getters* y *setters* para acceder a estos atributos.

```
36 public class CuentaBancaria implements Serializable {
37
38     // ATRIBUTOS -----
39
40     // De instancia
41     private static final long serialVersionUID = 1L;
42     private int numeroCuenta;
43     private int saldo;
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142     // GETTERS Y SETTERS -----
143
144
145     public int getNumeroCuenta() {
146
147         return numeroCuenta;
148     }
149
150
151     public void setNumeroCuenta(int numeroCuenta) {
152
153         this.numeroCuenta = numeroCuenta;
154     }
155
156
157     public int getSaldo() {
158
159         return saldo;
160     }
161
162
163     public void setSaldo(int saldo) {
164
165         this.saldo = saldo;
166     }
```

2. Package:

La accesibilidad de paquete permite que los atributos y métodos sean accesibles solo dentro del mismo paquete, promoviendo una mejor organización y modularidad del código.

Las constantes (**En las líneas de la 33 a la 38**) y atributos (**En las líneas 44, 45, y 46**) definidos en la enumeración **TipoTransporte** en el paquete **gestorAplicacion.externo** tienen accesibilidad de *paquete*, aunque su constructor es *privado*, para evitar que se creen más valores a los definidos en ella.

```

30 public enum TipoTransporte {
31
32     //CONSTANTES -----
33     CAMINANDO(1000, 5, "Caminando"),
34     BICICLETA(2000, 8, "Bicicleta"),
35     MOTO(5000, 10, "Moto"),
36     CARRO(10000, 30, "Carro"),
37     CAMION(50000, 100, "Camion"),
38     AVION(500000, 1000, "Avion");
39
40
41     // ATRIBUTOS -----
42
43     // De instancia
44     double precioEnvio_COP; //pesos colombianos
45     double capacidad_KG; //kilogramos
46     String nombre;

```

3. Protected:

Los atributos y métodos protected son accesibles dentro de su propio paquete y por las subclases, facilitando la herencia y la reutilización del código.

Tanto los atributos (**De la línea 34 a la 46**) como el constructor (**En la línea 63**) de la clase **Persona** tienen accesibilidad *protected*, ya que estos solo serán utilizados por las clases hijas (**Operario, Transportador y Vendedor**).

```

29 public abstract class Persona implements Serializable{
30
31     // ATRIBUTOS -----
32
33     // De clase
34     protected static final long serialVersionUID = 1L; // Versión del serializado asociada a esta clase
35     protected static int totalPersonas;
36     protected static ArrayList<Persona> listaPersonas = new ArrayList<Persona>();
37     protected static final int SALARIO = 50000;
38
39     // De instancia
40     protected String nombre;
41     protected int edad;
42     protected int identificacion;
43     protected CuentaBancaria cuentaBancaria;
44     protected int trabajado;
45     protected double minimoMeta;
46     protected ArrayList<Boolean> cumplimientoMetas = new ArrayList<Boolean>();

```

```

63 protected Persona(String nombre, int edad, int identificacion, CuentaBancaria cuentaBancaria) {
64     // Constructor con protected por ser una clase abstracta
65     this.nombre = nombre;
66     this.edad = edad;
67     this.identificacion = identificacion;
68     this.cuentaBancaria = cuentaBancaria;
69     this.trabajado = 0;
70     this.minimoMeta = 0;
71     this.cumplimientoMetas.add(false);
72     this.cumplimientoMetas.add(false);
73     listaPersonas.add(this);
74     totalPersonas++;
75 }
76
77
78 /**
79  * Constructor que no recibe parámetros.
80  */
81
82 protected Persona() {}

```

4. Public:

Los métodos públicos permiten que las funcionalidades de la clase sean accesibles desde cualquier parte del programa, facilitando su utilización.

Todos los métodos tienen accesibilidad pública, para facilitar su acceso. Un ejemplo de esto se puede ver en los métodos de la clase **Transporte**, algunos de estos son:

```

94 public void suministrarProducto(Tienda tienda, ArrayList<Producto> listaProductos) {
95
96     this.setTienda(tienda);
97     this.setListaProductos(listaProductos);
98 }

```

```

111 public static void MostrarTipoTransporte(String[] args) {
112
113     TipoTransporte[] transportes = TipoTransporte.values();
114
115     for (int i = 0; i < transportes.length; i++) {
116         System.out.println(transportes[i]);
117     }
118
119 }

```

```

131 public String TipoTransporte(TipoTransporte transporte) {
132
133     String mensaje = ("Tipo de transporte: " + transporte.name() +
134                     "Precio: " + transporte.getPrecioEnvio_COP() +
135                     "Capacidad máxima: " + transporte.getCapacidad_KG());
136
137     return mensaje;
138 }

```

```

152 public static Transporte enviarGratis(Transporte transporte){
153
154     transporte.getTipo().setPrecioEnvio_COP(0);
155
156     return transporte;
157 }

```

- Sobrecarga de métodos

La sobrecarga permite definir múltiples métodos con el mismo nombre, pero con diferentes parámetros.

- **Uso:** Proporciona flexibilidad para crear métodos con el mismo nombre pero que varíe según los parámetros que se le ingresen.
- **Beneficios y mejoras:** Permite utilizar un mismo nombre de método para diferentes tipos de parámetros, adaptándose a distintas necesidades. Adicionalmente, mejora la legibilidad del código al reducir la necesidad de nombres de métodos distintos para funciones similares. Por último, fomenta la reutilización del código al permitir que un mismo método maneje diferentes tipos de datos.
- **Ubicación:** Dentro del programa se realiza una sobrecarga de métodos para tres métodos de la clase **Factura** en el paquete **gestorAplicacion.empresa**. En estos se busca que a un método se le puedan ingresar diferentes parámetros y según esto se defina su comportamiento.

1. Método *gananciasPorDia()*:

En la **línea 180** se define por primera vez con dos parámetros de tipo entero y en la **línea 218** se define la sobrecarga con un parámetro de tipo ArrayList

```

180 public static ArrayList<Parejas<Integer, Double>> gananciasPorDia(int inicio, int fin) {

218 public static ArrayList<Parejas<Integer, Double>> gananciasPorDia(ArrayList<Integer> fechas) {

```


2. Método *obtenerValor()*:

En la **línea 310** se define por primera vez con dos parámetros de tipo `ArrayList` y entero, y en la **línea 405** se define la sobrecarga con dos parámetros de tipo `ArrayList` y `String`.

```
310 public static double obtenerValor(ArrayList<Parejas<Integer, Double>> lista, int clave) {  
  
405 public static Moda obtenerValor(ArrayList<Parejas<String, Moda>> lista, String clave) {
```

3. Método *listaFechas()*:

En la **línea 506** se define por primera vez con dos parámetros de tipo entero, y en la **línea 528** se define la sobrecarga sin parámetros.

```
506 public static ArrayList<Integer> listaFechas(int inicio, int fin){  
  
528 public static ArrayList<Integer> listaFechas(){
```

- Sobrecarga de constructores

Los constructores sobrecargados permiten crear objetos de diferentes maneras, mejorando la usabilidad y la reutilización del código.

- **Uso:**

1. Proporciona flexibilidad para crear objetos de tipo **Fabrica** con diferentes conjuntos de parámetros, permitiendo crear la fábrica con los atributos *listaProductos*, *listaTienda*, *cuentaBancaria* y *operario*, o crearla sin el *operario* o crearla sin pasar ningún valor.
2. Proporciona flexibilidad para crear objetos de tipo **Producto** con diferentes conjuntos de parámetros, permitiendo crear un producto con los atributos *nombre*, *descripcion*,

valor, peso, tamano, costoProduccion y categoría, o crearlo sin la *descripcion* o crearlo sin pasar ningún valor.

- **Beneficios y mejoras:** Permite inicializar objetos de múltiples formas, adaptándose a diferentes escenarios, facilitando la reutilización de código al permitir inicializaciones comunes en distintos constructores.
- **Ubicación:**
 1. En el paquete **gestorAplicacion.empresa** en la clase **Fabrica** en la **línea 66** se define el primer constructor, en la **línea 85** se define otro constructor con diferentes parámetros y en la **línea 99** se define un constructor sin parámetros.

```
66 public Fabrica(ArrayList<Producto> listaProductos, ArrayList<Tienda> listaTienda, CuentaBancaria cuentaBancaria, Operario operario) {
```

```
85 public Fabrica(ArrayList<Producto> listaProductos, ArrayList<Tienda> listaTienda, CuentaBancaria cuentaBancaria) {
```

```
99 public Fabrica() {}
```

2. En el paquete **gestorAplicacion.empresa** en la clase **Producto** en la **línea 66** se define el primer constructor, en la **línea 95** se define otro constructor con diferentes parámetros y en la **línea 104** se define un constructor sin parámetros.

```
66 public Producto(String nombre, String descripcion, double valor, double peso, double tamano, double costoProduccion, String categoria) {
```

```
95 public Producto(String nombre, double valor, double peso, double tamano, String categoria) {
```

```
104 public Producto() {}
```

- Manejo de referencias **this** para desambiguar y **this ()**

this se utiliza para referenciar la instancia actual de la clase y para desambiguar los nombres de los atributos. **this()** se usa para llamar a otro constructor dentro de la misma clase, permitiendo la reutilización de código en diferentes constructores.

1. this para desambiguar

- **Uso:** En los constructores se usa el **this** para desambiguar los nombres de los atributos con los nombres de los parámetros, un ejemplo de esto lo podemos ver en las clases **Cliente**, **Meta** y **Transporte**.
- **Beneficios y mejoras:** Mejora la claridad del código al hacer explícita la referencia a los atributos de la clase, eliminando las ambigüedades que puedan generarse.
- **Ubicación:**
 1. En el paquete **gestorAplicacion.externo** en la clase **Cliente** en las líneas 61, 62 y 63

```
59 public Cliente(String nombre, String direccion, CuentaBancaria cuentaBancaria) {  
60  
61     this.nombre = nombre;  
62     this.direccion = direccion;  
63     this.cuentaBancaria = cuentaBancaria;  
64     listaClientes.add(this);  
65 }
```

2. En el paquete **gestorAplicacion.empleados** en la clase **Meta** en las líneas 49, 50 y 51

```
47 public Meta(String nivel, double minimo, double comision) {  
48  
49     this.nivel = nivel;  
50     this.minimo = minimo;  
51     this.comision = comision;  
52 }
```

3. En el paquete **gestorAplicacion.externo** en la clase **Transporte** en las líneas 73, 74, 75 y 76

```

71 public Transporte(TipoTransporte tipo, Double capacidad, double costo, Transportador transportador) {
72
73     this.tipo = tipo;
74     this.capacidad = capacidad;
75     this.costo = costo;
76     this.transportador = transportador;
77 }

```

2. this()

- **Uso:**

1. En la clase **Fabrica**, en el primer constructor se llama a **this** (*listaProductos*, *listaTienda*, *cuentaBancaria*), para reutilizar el código del segundo constructor, y luego inicializa el valor de *operario*.
2. En la clase **Producto**, en el segundo constructor se llama al otro constructor de la clase dando a los atributos *descripcion* y *costoProduccion* un valor por defecto.

- **Beneficios y mejoras:** Fomenta la reutilización de código al permitir que un constructor llame a otro, reduciendo la duplicación de código.

- **Ubicación:**

1. En el paquete **gestorAplicacion.empresa** en la clase **Fabrica** en la **línea 69** se hace uso de **this()**.

```

66 public Fabrica(ArrayList<Producto> listaProductos, ArrayList<Tienda> listaTienda, CuentaBancaria cuentaBancaria, Operario operario) {
67
68     // Se llama al otro constructor que recibe estos parametros
69     this(listaProductos, listaTienda, cuentaBancaria);
70     // Se define el atributo que no esta en el otro constructor
71     this.operario = operario;
72 }
73

```

2. En el paquete **gestorAplicacion.empresa** en la clase **Producto** en la **línea 96** se hace uso de **this()**.

```

95 public Producto(String nombre, double valor, double peso, double tamano, String categoria){
96     this(nombre, "Sin descripción", valor, peso, tamano, 5.000, categoria);
97 }

```

- Caso de enumeración

Una enumeración (enum) en programación es una estructura de datos que permite definir un conjunto fijo de constantes con nombre y métodos relacionados a estas.

- **Uso: TipoTransporte** es una enumeración que define diferentes tipos de transporte con sus características asociadas como precio de envío (*precioEnvio_COP*), capacidad en kilogramos (*capacidad_KG*), y nombre del transporte (*nombre*). Cada constante dentro de la enumeración (como CAMINANDO, BICICLETA, etc.) se instancia con valores específicos usando un constructor privado.
- **Beneficios y mejoras:** Aumenta la seguridad del código al restringir los valores posibles a un conjunto definido de constantes y facilita la lectura del código, y por tanto su escritura.
- **Ubicación: TipoTransporte.java** es una enumeración donde se definen tipos de transporte con valores fijos y métodos específicos para los tipos de transporte definidos aquí, esta enumeración se encuentra en el paquete **gestorAplicacion.externo**

```
30 public enum TipoTransporte {
31
32     //CONSTANTES -----
33     CAMINANDO(1000, 5, "Caminando"),
34     BICICLETA(2000, 8, "Bicicleta"),
35     MOTO(5000, 10, "Moto"),
36     CARRO(10000, 30, "Carro"),
37     CAMION(50000, 100, "Camion"),
38     AVION(500000, 1000, "Avion");
39
40
41     // ATRIBUTOS -----
42
43     // De instancia
44     double precioEnvio_COP; //pesos colombianos
45     double capacidad_KG; //kilogramos
46     String nombre;
47 }
```

Algunos de los métodos definidos son:

```

84 public static ArrayList<TipoTransporte> transporteSegunCarga(int PesoTotalProductos) {
85
86     ArrayList<TipoTransporte> transportesPosibles = new ArrayList<TipoTransporte>();
87
88     for(TipoTransporte tipoTransporte : TipoTransporte.values()) {
89
90         if(tipoTransporte.getCapacidad_KG() >= PesoTotalProductos){
91             transportesPosibles.add(tipoTransporte);
92         }
93     }
94
95     return transportesPosibles;
96 }
97

```

```

158 @Override
159 public String toString() {
160     return this.getNombre();
161 }
162

```

4. Descripción de funcionalidades

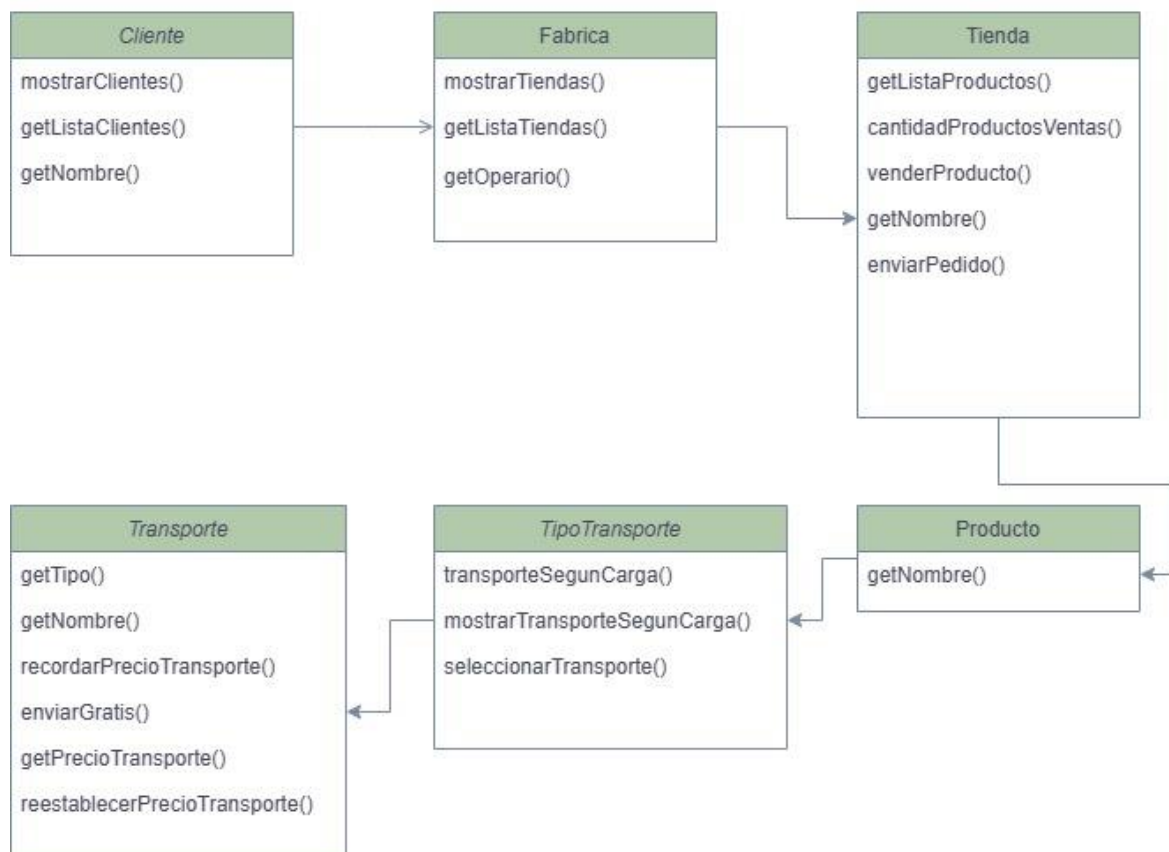
❖ Funcionalidad 1: Envío pedidos

Permite seleccionar productos desde una tienda y enviarlos a un cliente usando un tipo de transporte adecuado.

Interacciones: **Cliente** selecciona productos (**Producto**) desde una **Tienda**, se elige un **Transporte**, y se genera una **Factura**.

Link: https://drive.google.com/file/d/1_D9lvEZ_w1uHpYUfIlRSWQL--mlemmQB/view?usp=sharing

Funcionalidad Envío Pedidos



La clase EnvioPedidos es parte del paquete **uiMain** y está diseñado para gestionar el envío de pedidos desde una tienda hacia un cliente. El funcionamiento de esta lo podemos describir en una serie de pasos que llevan a que la funcionalidad cumpla el objetivo.

Al comienzo de la funcionalidad **envioPedidos**, se inicializan los atributos necesarios para la funcionalidad. Primero, se definen las variables *elección* y *x* para manejar la selección de opciones en el menú. También se inicializa *interruptor* como *true* para mantener el bucle principal en ejecución hasta que se indique lo contrario. Luego, se crean dos objetos Scanner, *sc* y *sc2*, para leer la entrada del usuario desde la consola. Posteriormente, se declaran los atributos *clienteSeleccionado*, *tiendaSeleccionada*, *productoSeleccionado*, *transporteSeleccionado*, y *tipoTransportes* para almacenar temporalmente las selecciones realizadas por el usuario durante el proceso. Por último, se crean los atributos complementarios como *PesoTotalProductos* para acumular el peso de los productos seleccionados, *numEnvioGratis* para manejar la opción de envío gratuito, y *listaProductosPedidos*, que es una lista donde se almacenan los productos seleccionados

Esta parte se puede observar en código de la siguiente manera:

```
37 // Atributos a usar
38 int x = 1; // para manejar el menu
39 int eleccion = 1;
40 int numProductoSeleccionado = 1;
41
42 Scanner sc = new Scanner(System.in);
43 Scanner sc2 = new Scanner(System.in);
44
45 Boolean interruptor = true; // Para manejar el ciclo while
46 Cliente clienteSeleccionado = null;
47 Tienda tiendaSeleccionada = null;
48 Producto productoSeleccionado = null;
49 Transporte transporteSeleccionado = null;
50 TipoTransporte tipoTransportes = null;
51 int PesoTotalProductos = 0;
52 int numEnvioGratis=0;
53 ArrayList<Producto> listaProductosPedidos = new ArrayList<Producto>();
```

El núcleo del flujo de la aplicación es un bucle *while* que continúa ejecutándose mientras *interruptor* sea true. Dentro de este bucle, el flujo de control se maneja mediante la variable *elección*, que determina la siguiente acción a tomar, de acuerdo con el número de la opción seleccionada por el usuario.

Esta parte se puede observar en código de la siguiente manera:


```
56      // Manejo de opciones
57      while (interruptor) {
58
59          switch (eleccion) {
60
```

Partiendo de aquí, continuamos con el desarrollo de la funcionalidad en una serie de pasos organizados:

1. Seleccionar cliente

El paso uno consiste en que el usuario debe seleccionar un cliente al que desea enviar un pedido. Para esto, se siguen los siguientes pasos:

1. Se muestra una lista de clientes disponibles utilizando el método *mostrarClientes()* de la clase **Cliente** (Definido en la línea 87)
2. El usuario introduce un número que corresponde al cliente deseado.
3. Si el número es válido, se asigna el cliente a la variable *clienteSeleccionado* y se avanza a la siguiente etapa de selección (*eleccion = 2*). Si el número no es válido se le notifica al usuario. Validamos que el número ingresado por el usuario sea válido mediante un condicional que verifica que el valor esté entre 1 y el valor que retorna el método *getListaCliente()* de la clase **Cliente** (Definido en la línea 192).

Esta parte se puede observar en código de la siguiente manera:

```

68     case 1: // Seleccionar cliente
69
70         System.out.print("\nSeleccione el cliente al que desea enviar: \n");
71         System.out.println("0. Volver al menu anterior \n");
72         System.out.println(Cliente.mostrarClientes());
73
74         // Entero seleccionado
75         System.out.print("> ");
76         int numClienteSeleccionado = sc.nextInt(); // Pregunta al usuario
77
78         if (numClienteSeleccionado == 0) { // Volver al menú anterior
79
80             eleccion = 0;
81             break;
82
83         }
84
85         if (numClienteSeleccionado > Cliente.getListaClientes().size()) { // Número de cliente mayor a la cantidad de clientes registrados
86
87             System.out.println("Número de cliente inválido, por favor seleccione un cliente en la lista");
88             eleccion = 1;
89             break;
90
91         } else {
92
93             clienteSeleccionado = Cliente.getListaClientes().get(numClienteSeleccionado - 1);
94             System.out.print("Has seleccionado al cliente #" + numClienteSeleccionado + "\nEl cliente es: "
95                 + clienteSeleccionado.getNombre());
96             eleccion = 2;
97
98         }

```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```

Seleccione el cliente al que desea enviar:
0. Volver al menu anterior

1. Esteban Carrillo Dirección: Calle 64 # 20
2. Tiffany Mendoza Dirección: Carrera 89 # 17
3. Santiago Perez Dirección: Avenida 35 # 21
4. Mariana Lopera Dirección: Carrera 23 # 56
5. Jose Manuel Vergara Dirección: Calle 65 # 21
6. Camila Zapata Dirección: Avenida 12 # 56
7. Miguel Acosta Dirección: Carrera 37 # 67
8. Martin Berrio Dirección: Calle 45 # 45

> 1
Has seleccionado al cliente #1
El cliente es: Esteban Carrillo

```

2. Seleccionar Tienda:

En el paso 2 el usuario selecciona la tienda desde la cual se enviará el pedido, siguiendo los siguientes pasos:

1. Se muestra una lista de tiendas disponibles utilizando el método *mostrarTiendas()* de la clase **Fabrica** (Definido en la línea 140), al cual accede desde el atributo *fabrica* de la clase **Cargar**.
2. El usuario selecciona una tienda ingresando un número.

3. Si el número es válido, se asigna la tienda seleccionada a la variable *tiendaSeleccionada* y se avanza a la selección de productos (*eleccion* = 3). Al igual que en la etapa anterior, con un condicional verificamos si el valor está entre 1 y lo que nos retorna el método *getListaTienda()* (Qué es el valor máximo admitido) de la clase **Fabrica** (Definido en la línea 313), al cual accede desde el atributo *fabrica* de la clase **Cargar**.

Esta parte se puede observar en código de la siguiente manera:

```
101         case 2: // seleccionar la tienda
102
103             System.out.println("\n");
104             System.out.println("Su pedido se enviará desde alguna de estas tiendas, por favor seleccione una:");
105             System.out.println("0. Volver al menú principal");
106             System.out.print(Cargar.fabrica.mostrarTiendas());
107
108             // Seleccionar tienda
109             System.out.println("Seleccione la tienda desde la que desea enviar: ");
110
111             // Entero seleccionado
112             System.out.print("> ");
113             int numTiendaSeleccionada = sc.nextInt();
114
115             if (numTiendaSeleccionada == 0) { // Volver al menú principal
116
117                 eleccion = 0;
118                 break;
119             }
120
121             else if (numTiendaSeleccionada > Cargar.fabrica.getListaTienda().size() || numTiendaSeleccionada < 0) {
122
123                 System.out.println("Número de tienda inválido, por favor seleccione una tienda en la lista");
124                 eleccion = 2;
125                 break;
126             }
127
128             } else { // Número ingresado es válido
129
130                 tiendaSeleccionada = Cargar.fabrica.getListaTienda().get(numTiendaSeleccionada - 1);
131                 System.out.println("Has seleccionado la tienda: " + numTiendaSeleccionada);
132                 eleccion = 3;
133             }
```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

Su pedido se enviará desde alguna de estas tiendas, por favor seleccione una:

0. Volver al menu principal

1. Delicia Fresca Norte-Productos:

Manzana: 4
Zanahoria: 4
Pepino: 4
Limon: 4
Piña: 4
Pera: 4
Pan: 4
Sanduche: 4
Pandebono: 4
Pan hojaldrado: 4
Galletas: 4
Palito de queso: 4
Mermelada de frambuesa: 4
Mermelada de piña: 4
Mermelada de mora: 4
Mayonesa: 4
Salsa de tomate: 4
Mermelada de fresa: 4
Te verde: 4
Jugo de naranja: 4
Leche de almendras: 4
Leche de avena: 4
Yogur de soya: 4
Cafe instantaneo: 4
Jugo de mora: 4

2. Delicia Fresca Sur-Productos:

Pera: 4
Tomate: 4
Frambuesa: 4
Lechuga: 4
Sandia: 4
Pan: 4
Palito de queso: 4
Tostada: 4
Ponque: 4
Pan tajado: 4
Almojabana: 4
Mermelada de frambuesa: 4
Mermelada de fresa: 4
Mermelada de kiwi: 4
Mermelada de frutos rojos: 4
Salsa BBQ: 4
Salsa de soya: 4
Te verde: 4
Cafe instantaneo: 4
Bebida achocolatada: 4
Leche de coco: 4
Bebida hidratante: 4
Jugo de mora: 4
Pepino: 4
Jugo de naranja: 4

```

3. Delicia Fresca Centro-Productos:
Manzana: 4
Zanahoria: 4
Limon: 4
Piña: 4
Tomate: 4
Frambuesa: 4
Lechuga: 4
Sandia: 4
Sanduche: 4
Pandebono: 4
Pan hojaldrado: 4
Galletas: 4
Tostada: 4
Ponque: 4
Pan tajado: 4
Almojabana: 4
Mermelada de piña: 4
Mermelada de mora: 4
Mayonesa: 4
Salsa de tomate: 4
Mermelada de kiwi: 4
Mermelada de frutos rojos: 4
Salsa BBQ: 4
Salsa de soya: 4
Leche de almendras: 4
Leche de avena: 4
Yogur de soya: 4
Bebida achocolatada: 4
Leche de coco: 4
Bebida hidratante: 4

Seleccione la tienda desde la que desea enviar:
>

```

3. Selección de productos

En el paso 3, el usuario selecciona los productos que desea comprar de la tienda seleccionada. Para esto, sigue estos pasos:

1. Se le pregunta al usuario cuántos productos desea comprar, con un límite máximo de 5 productos por cliente.
2. Se verifica que el número de productos seleccionado esté dentro del rango permitido y que la tienda tenga suficientes productos disponibles.

Esta parte se puede observar en código de la siguiente manera:

```

137     case 3: // seleccionar el producto
138
139         System.out.println("¿Cuántos productos deseas comprar de esta tienda? \n Máximo 5 productos por cliente");
140         int cuantos = sc.nextInt();
141
142         if (cuantos > 5 || cuantos < 0) { // Número fuera del rango entre 0 y 5
143
144             System.out.println("No es válido, elija un numero menor o igual a 5");
145             eleccion = 3;
146             break;
147         }
148
149
150         if (cuantos > tiendaSeleccionada.getListaProductos().size()) { // Se quieren comprar más productos de los que estan disponibles
151
152             System.out.println("***La tienda de la que quieres comprar solo tiene " + tiendaSeleccionada.getListaProductos().size() +
153             ". Entonces te dejaremos comprar " + tiendaSeleccionada.getListaProductos().size() + " productos.");
154
155             cuantos = tiendaSeleccionada.getListaProductos().size();
156
157         }
158

```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```

Has seleccionado la tienda: 1
¿Cuántos productos deseas comprar de esta tienda?
Máximo 5 productos por cliente
1

Seleccione el producto que desea enviarle al cliente
0. Regresar al menu principal

```

3. El usuario selecciona cada producto de la lista mostrada por la tienda. Los productos seleccionados se añaden a la lista *listaProductosPedidos*, y se actualiza el peso total de los productos seleccionados (*PesoTotalProductos*).

Si se selecciona un número válido para el producto (valor entre 1 y lo que nos retorna *getListaProductos()*) y se seleccionó el número de productos deseados, el flujo avanza a la etapa de selección de transporte (*eleccion* = 4).

Esta parte se puede observar en código de la siguiente manera:

```

159     if (cuantos <= 5 && cuantos > 0) { // Número dentro del rango de 0 a 5 (Rango permitido)
160
161         for (int i = 0; i < cuantos; i++){
162
163             System.out.println("\nSelecione el producto que desea enviarle al cliente");
164             System.out.println("0. Regresar al menu principal");
165             System.out.println(tiendaSeleccionada.cantidadProductosVentas());
166             System.out.print("> ");
167             numProductoSeleccionado = sc.nextInt(); // Se pregunta al usuario
168             // Se establece el intervalo en el que estan los productos
169
170             if (numProductoSeleccionado == 0) { // Volver al menú anterior
171
172                 eleccion = 0;
173                 break;
174             }
175
176             if (numProductoSeleccionado > tiendaSeleccionada.getListaProductos().size() || numProductoSeleccionado < 0) { // Una condición o la otra (ó) - C
177
178                 System.out.println("Número de producto inválido, por favor seleccione un producto en la lista");
179                 cuantos++;
180
181             } else { // El número ingresado es válido
182
183                 productoSeleccionado = tiendaSeleccionada.getListaProductos().get(numProductoSeleccionado - 1);
184                 System.out.print("Ha seleccionado el producto # " + numProductoSeleccionado + " Nombre del producto: " + productoSeleccionado.getNombre());
185                 listaProductosPedidos.add(productoSeleccionado);
186                 tiendaSeleccionada.venderProducto(productoSeleccionado);
187                 pesoTotalProductos += productoSeleccionado.getPeso();
188                 eleccion = 4;
189             }
190
191         }

```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```

Ha seleccionado el producto # 1 Nombre del producto: Pepino

```

4. Seleccionar Transporte:

Una vez que los productos han sido seleccionados, continuamos al paso 4 donde el usuario elige el tipo de transporte para enviar el pedido, siguiendo estos pasos:

1. Se muestra una lista de transportes disponibles, filtrada por aquellos que pueden soportar el peso total de los productos seleccionados, utilizando el método *transporteSegunCarga()* de la Enumeración **TipoTransporte** (Definido en la línea 84).
2. El usuario selecciona un transporte ingresando un número.
3. Si el número es válido (verificando si está entre 1 y el máximo que nos retorna el tamaño de la lista de los transportes filtrados), el transporte se asigna al atributo *transporteSeleccionado*.

Esta parte se puede observar en código de la siguiente manera:

```

198     case 4: // seleccionar tipo de transporte
199
200         System.out.println("\n\nSeleccione en que medio de transporte quiere enviar este producto");
201         System.out.println("Junto a cada tipo de transporte se encuentra su precio.");
202         System.out.println(
203             "\nAdvertencia: Los tipos de transporte han sido filtrados de manera que solo puede seleccionar los que puedan soportar el peso de su pro
204             + "\nSu pedido pesa " + PesoTotalProductos + " kilogramos");
205
206         System.out.println("0. Regresar al menu principal");
207
208         // Encontrar los transportes posibles
209         ArrayList<TipoTransporte> listaTransFiltrada = new ArrayList<TipoTransporte>();
210         listaTransFiltrada = TipoTransporte.transporteSegunCarga(PesoTotalProductos);
211         System.out.println(TipoTransporte.mostrarTransporteSegunCarga(listaTransFiltrada));
212         System.out.println("Seleccione el número del tipo de transporte que desea: ");
213         System.out.print("> ");
214         int numTransporteSeleccionado = sc.nextInt();
215
216         if (numTransporteSeleccionado == 0) { // Volver al menú anterior
217
218             eleccion = 0;
219             break;
220
221         }
222
223         if (numTransporteSeleccionado > listaTransFiltrada.size() || numTransporteSeleccionado < 0) { // Una condición o la otra (ó) - Cuando el número i
224
225             System.out.println("Número de transporte inválido, por favor seleccione un producto en la lista");
226             eleccion = 4;
227             break;
228
229         } else { // El número ingresado es válido
230
231             transporteSeleccionado = TipoTransporte.seleccionarTransporte(listaTransFiltrada, numTransporteSeleccionado);
232             System.out.print("Ha seleccionado el transporte #" + (numTransporteSeleccionado)
233                 + "\nEl pedido se enviará por " + transporteSeleccionado.getTipo().getNombre());
234             System.out.println("\nDesea aplicar envío gratis?\n 1. Si   2. No");
235             System.out.print("> ");
236             transporteSeleccionado.recordarPrecioTransporte();

```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```

Seleccione en que medio de transporte quiere enviar este producto
Junto a cada tipo de transporte se encuentra su precio.

Advertencia: Los tipos de transporte han sido filtrados de manera que solo puede seleccionar los que puedan soportar el peso de su producto.
Su pedido pesa 0 kilogramos
0. Regresar al menu principal
1. Caminando $1000.0
2. Bicicleta $2000.0
3. Moto $5000.0
4. Carro $10000.0
5. Camion $50000.0
6. Avion $500000.0

Seleccione el número del tipo de transporte que desea:
> 1
Ha seleccionado el transporte #1
El pedido se enviará por Caminando

```

4. El usuario tiene la opción de aplicar un envío gratis, lo que reduce el costo del transporte a \$0. Esta opción se maneja mediante la variable numEnvioGratis.

Esta parte se puede observar en código de la siguiente manera:


```

238 // Aplicar envío gratis para el pedido
239 while(true){
240
241     numEnvioGratis = sc.nextInt();
242
243     if (numEnvioGratis == 1){
244
245         Transporte.enviarGratis(transporteSeleccionado);
246         System.out.println("Su tarifa de envío ha bajado de: $" + transporteSeleccionado.getPrecioTransporte() + " a $0");
247         break;
248
249     }else if(numEnvioGratis != 2){
250
251         System.out.println("Seleccione un numero dentro del rango");
252         System.out.print("> ");
253
254     }else{
255
256         break;
257
258     }
259 }

```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```

Desea aplicar envío gratis?
1. Si    2. No

```

5. Una vez seleccionado el transporte y aplicada (o no) la opción de envío gratis, el flujo avanza a la generación de la factura (*eleccion* = 5).

5. Generación de la Factura

Finalmente, como último paso se genera la factura del pedido y se finaliza el proceso, siguiendo estos pasos:

1. Se solicita al usuario que ingrese el día del mes en que desea que se realice el envío.
2. Se genera una factura que incluye detalles como la tienda, el cliente, los productos seleccionados, y el transporte, usando el método *enviarPedido()* de la clase **Tienda** (Definido en la línea 211).
3. Si se aplica un envío gratis, se restablece el precio original del transporte, usando el método *reestablecerPrecioTransporte()* de la clase **Transporte** (Definido en la línea 180).
4. Se le pregunta al usuario si desea realizar otro envío o regresar al menú principal. Dependiendo de su elección, se reinicia el proceso de selección o se sale del bucle principal.

Esta parte se puede observar en código de la siguiente manera:

```

267 case 5: // retorna factura
268
269 System.out.println("\nDigite el día del mes entre 1 y 30: ");
270 System.out.print("> ");
271 int dia = 1;
272
273 while(true){
274
275     dia = sc.nextInt();
276
277     if (dia >= 1 && dia <=30){
278
279         break;
280
281     }else{
282
283         System.out.println("Seleccione un numero dentro del rango");
284         System.out.print("> ");
285
286     }
287 }
288
289 System.out.println("\n-----\n" + "Factura generada en la tienda "
290 +tiendaSeleccionada.getNombre() + "\nA nombre del cliente: " + clienteSeleccionado.getNombre()+"\n"
291 +tiendaSeleccionada.enviarPedido(new ArrayList<>(listaProductosPedidos),
292     transporteSeleccionado, clienteSeleccionado, dia, Cargar.fabrica.getOperario())+"\n-----\n\n\n");
293
294 if(numEnvioGratis==1){
295
296     transporteSeleccionado.reestablecerPrecioTransporte();
297
298 }
299
300 listaProductosPedidos.clear();
301
302 // Realiza una copia de la listaProductosPedidos
303 PesoTotalProductos = 0;
304 System.out.println("¿Desea hacer otro envío o volver al menú principal? ");
305 System.out.println("0. Volver al menú principal");
306 System.out.println("1. Realizar otro envío");
307 System.out.print("> ");
308 int opcion = sc2.nextInt();
309
310 while (true) {
311
312     if (opcion == 0) {
313
314         eleccion = 0;
315         interruptor = false;
316         break;
317
318     } else if (opcion == 1) {
319
320         eleccion = 1;
321         break;
322
323     } else {
324
325         System.out.println(" La opcion que digitó es incorrecta ");

```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```

Digite el día del mes entre 1 y 30:
> 1

-----
Factura generada en la tienda Delicia Fresca Norte
A nombre del cliente: Esteban Carrillo
Nombre del producto: Pepino - Precio del producto: 1500.0
Tipo de transporte:          CAMINANDO
Tarifa de envío:             0.0
Total a pagar:               1500.0
-----

¿Desea hacer otro envío o volver al menú principal?
0. Volver al menú principal
1. Realizar otro envío

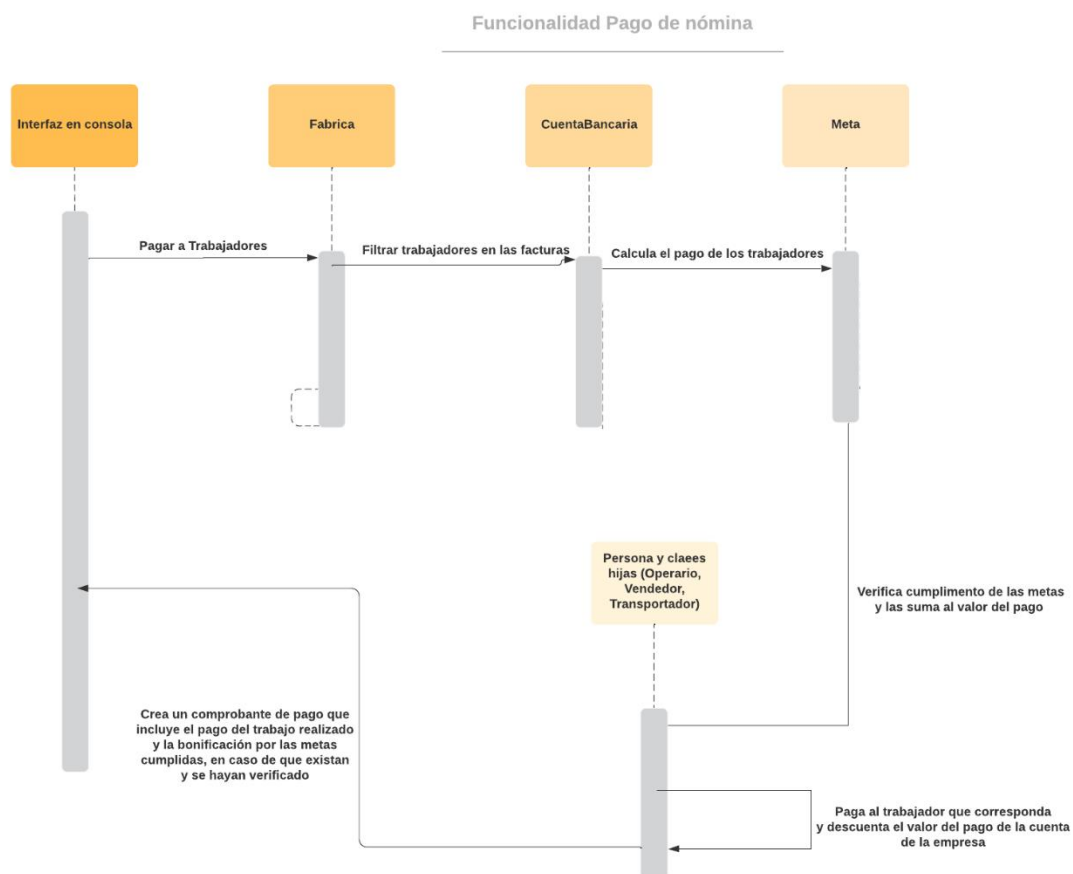
```

❖ Funcionalidad 2: Pago de nómina

Esta funcionalidad implementa la lógica para gestionar el pago de la nómina a los trabajadores de la empresa (operarios, transportadores, vendedores), en base a sus metas alcanzadas. Estas metas se miden teniendo en cuenta el trabajo que ha realizado cada trabajador en los envíos. Para los vendedores y operarios se toma en cuenta el numero de productos vendidos y para los transportadores se toma en cuenta el peso de los productos que ha llevado.

Interacciones: **Fabrica** y **Factura** proporcionan información acerca de los empleados (**Operario**, **Transportador**, **Vendedor**), a quienes se les verificarán sus **Metas** y se les pagará lo debido en su **CuentaBancaria**. De la siguiente manera:

Link: <https://lucid.app/lucidchart/8bed67f2-c9f6-4a76-89d1-47c713d35967/view>



La funcionalidad está construida en una serie de pasos para poder realizar el pago a los trabajadores de manera correcta, teniendo en cuenta el trabajo que han realizado y las metas que han cumplido.

Como primero, se confirma que se seleccionó la opción de pago a trabajadores. También, se preguntará a qué tipo de trabajador se le quiere pagar. Para esto, se muestra un menú (utilizando la clase **MenuAuxiliar**) que permite al usuario seleccionar el tipo de trabajador al que desea pagar: **Operarios**, **Transportadores** o **Vendedores**. También se da la opción de volver al menú principal.

Esta parte se puede observar en código de la siguiente manera:

```
46     while (verificador1) {
47
48         int opcSalida;
49         boolean verificador2 = true;
50         boolean verificador3 = true;
51
52         int opcion = new MenuAuxiliar("\n¿A qué tipo de trabajador desea pagarle?",
53             new String[] { "Operarios", "Transportadores", "Vendedores" },
54             "Volver al menú principal").mostrar();
55
56         if (opcion == 0) {
57
58             verificador1 = false;
59             break;
60         }
```

Como podemos ver, en caso de elegir la opción 0, el programa cerrará automáticamente el ciclo, volviendo al menú principal. También, es importante destacar que tanto en esta como en las demás elecciones del usuario se realizan verificaciones para que la opción seleccionada esté dentro del rango requerido.

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```
Has escogido la opción pagar a trabajadores
¿A qué tipo de trabajador desea pagarle?
1. Operarios
2. Transportadores
3. Vendedores
0. Volver al menú principal
```

Luego de hacer la selección, se guarda en el atributo *listaTrabajadores* el resultado de ejecutar el método *TrabajadoresInvolucrados()* de la clase **Fabrica** (Definido en la línea 259) para obtener una lista de trabajadores del tipo seleccionado que han estado involucrados en las facturas creadas. Al método *TrabajadoresInvolucrados()* se le pasan como parámetros la lista de facturas obtenida del método *getListaFacturas()* de la clase **Factura** (Definido en la línea 708) y la opción seleccionada por el usuario.

Si *listaTrabajadores* está vacía (Sea porque no hay pagos pendientes o no se han realizado envíos), se muestra un mensaje indicando que no hay trabajadores de ese tipo que hayan trabajado o tengan pagos pendientes, y se rompe el ciclo para preguntar si desea pagar a otro trabajador o volver al menú principal.

Esta parte se puede observar en código de la siguiente manera:

```
62         while (verificador3) {
63
64             // Método #1 Verifica los trabajadores del tipo que seleccionó
65             // salen en las facturas que han sido creadas
66             ArrayList<Persona> listaTrabajadores = Fabrica.TrabajadoresInvolucrados(Factura.getListaFacturas(), opcion);
67
68             if (listaTrabajadores.size() == 0) {
69
70                 System.out.print("\nLo sentimos, ningun de trabajador de este tipo ha trabajado por el momento\n");
71                 verificador3 = false;
72                 break;
73             }
74         }
75     }
```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```
Lo sentimos, ningun de trabajador de este tipo ha trabajado por el momento
¿Qué desea hacer?
1.Pagar a otro trabajador
0.Volver al menu principal
```

Por otro lado, si *listaTrabajadores* no está vacía, se muestra un mensaje indicando que se mostrará la lista de este tipo de trabajadores que han trabajado. Aquí, el mensaje que se muestra depende del tipo de trabajador seleccionado (**Operarios**, **Transportadores** o **Vendedores**).

Esta parte se puede observar en código de la siguiente manera:

```

75 // Manejo de opciones
76 switch (opcion) {
77
78     case 1:
79
80         System.out.println("\nEsta es la lista de Operarios que han trabajado");
81         break;
82
83     case 2:
84
85         System.out.println("\nEsta es la lista de Transportadores que han trabajado");
86         break;
87
88     case 3:
89
90         System.out.println("\nEsta es la lista de Vendedores que han trabajado");
91         break;
92
93 }
94

```

Luego de esto, se muestra la lista de aquellos trabajadores del tipo seleccionado que han trabajado y tienen pagos pendientes, esta lista se muestra utilizando el método *mostrarEmpleados()* de la clase **Fabrica** (Definido en la línea 206).

Esta parte se puede observar en código de la siguiente manera:

```

96 // Imprime la lista que se devolvió anteriormente
97 System.out.println(Fabrica.mostrarEmpleados(listaTrabajadores));

```

Por último, se pide que ingrese el número del trabajador al que desea pagarle y se especifica el rango válido para ingresar, yendo desde uno, hasta el tamaño de la lista *listaTrabajadores*.

Esta parte se puede observar en código de la siguiente manera:

```

99 while (verificador2) {
100     // Pide el # del trabajador
101     if (listaTrabajadores.size() == 1) {
102
103         System.out.println("Por favor, ingrese el número del trabajador al cual desea pagarle [1]: ");
104
105     } else {
106
107         System.out.println("Por favor, ingrese el número del trabajador al cual desea pagarle [1-"
108             + listaTrabajadores.size() + "]: ");
109     }
110
111     System.out.print("> ");

```

Adicionalmente, podemos ver los tres pasos anteriores en la consola al ejecutar el código de la siguiente manera:

```

Esta es la lista de Vendedores que han trabajado

Trabajador 1
Nombre:      Isabella Restrepo
Edad:        19
Cedula:      2222222
Tienda:      Delicia Fresca Norte

Trabajador 2
Nombre:      Carlos Escobar
Edad:        21
Cedula:      4444444
Tienda:      Delicia Fresca Centro

Por favor, ingrese el número del trabajador al cual desea pagarle [1-2]:

```

Aquí, se le pide al usuario que ingrese el número del trabajador al que desea pagar.

Si el número ingresado es válido, se procede a obtener el trabajador seleccionado de la lista y guardarlo en el atributo de *trabajadorEscogido* de tipo **Persona**.

En este caso, seleccionamos a un trabajador de tipo Vendedor, pero funciona igual para los otros dos tipos.

Esta verificación se ve en el código de la siguiente manera:

```

114         if (opcTrabajador <= 0 || opcTrabajador > listaTrabajadores.size()) {
115
116             System.out.println("Opción de trabajador incorrecta");
117
118         } else {
119
120             Persona trabajadorEscogido = listaTrabajadores.get(opcTrabajador - 1);
121

```

Luego, se calculará el pago básico que se le debe al trabajador utilizando el método *calcularPago()* de la clase **CuentaBancaria** (Definido en la línea 102) y se guardará en el atributo *total*.

Después, se muestra el nombre del trabajador (Usando el método *getNombre()* de la clase **Persona** (Definido en la línea 136), seguido del pago calculado (atributo *total*) y la cantidad de veces que el trabajador ha trabajado (Usando el método *getTrabajado()* de la clase **Persona** (Definido en la línea 184).

Luego, se pregunta al usuario si desea analizar y bonificar al trabajador por metas las cumplidas.

Esta parte se puede observar en código de la siguiente manera:


```

122         // Método #2
123         // Calcula el pago que se le va a dar al trabajador escogido
124         int total = CuentaBancaria.calcularPago(trabajadorEscogido);
125
126         System.out.println("\nAl trabajador " + trabajadorEscogido.getNombre() + " se le pagará " + total
127             + " por trabajar " + trabajadorEscogido.getTrabajado() + " veces");
128
129         System.out.println(
130             "\n¿Desea analizar y bonificar al trabajador por sus metas cumplidas?\n1.Si\n2.No");
131
132         boolean verificador4 = true;
133
134         int valorPorMetas = 0;
135
136         ArrayList<Integer> indicesMetasMostradas = new ArrayList<Integer>();
137

```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```

> 2
Al trabajador Carlos Escobar se le pagará 70000 por trabajar 1 veces
¿Desea analizar y bonificar al trabajador por sus metas cumplidas?
1.Si
2.No
Digite el número de la opción que desee

```

Si el usuario selecciona que sí desea bonificar al trabajador, se procede a mostrar las metas disponibles según el tipo de trabajador. Se permite al usuario seleccionar una meta y verificar si se ha cumplido. Si la meta ha sido cumplida, se añade la bonificación al total del pago.

Este comportamiento se realiza con el método *cumplioMeta()* de la clase **Meta** (Definido en la línea 69), que verifica si la meta ha sido cumplida con base en un índice específico, y el método *porcentajeCumplimiento()* de la clase **Meta** (Definido en la línea 87), que devuelve estadísticas sobre el cumplimiento de la meta.

Esta parte se puede observar en código de la siguiente manera:

1. Se accede a la información de las metas usando el tipo específico del empleado.

```

153         case 1:
154
155             System.out.println("\nEl trabajador escogido tiene las siguientes metas\n");
156
157             ArrayList<Meta> listaMetas;
158             String textoIndice;
159             double indice = trabajadorEscogido.getMinimoMeta();
160
161             if (opcion==1){
162
163                 listaMetas = Operario.getMetasOperario();
164                 textoIndice = "Indice: número de productos vendidos\n";
165
166             }else if(opcion==2){
167
168                 listaMetas = Transportador.getMetas();
169                 textoIndice = "Indice: peso de productos transportados\n";
170
171             }else{
172
173                 listaMetas = Vendedor.getMetas();
174                 textoIndice = "Indice: número de productos vendidos\n";
175
176             }
177
178             System.out.println(textoIndice);
179

```

2. Verifica que la meta seleccionada no haya sido mostrada previamente. Si lo ha sido, lo informa y si ya fue cumplida informa que ya ha sido pagada.

```

194         if (indicesMetasMostradas.contains(opcMetaEscogida)){
195
196             System.out.println("Esta opción ya fue mostrada");
197
198         }else if(trabajadorEscogido.getCumplimientoMetas().get(opcMetaEscogida - 1)==true){
199
200             System.out.println("La meta que seleccionó ya fue cumplida y se le ha pagado al trabajador su bonificación");
201

```

3. Si la meta seleccionada no ha sido revisada, procede a revisar si esta meta fue cumplida por el empleado

```

209         }else{
210
211             Meta metaEscogida = listaMetas.get(opcMetaEscogida - 1);
212             boolean verificadorMeta = metaEscogida.cumplicMeta(indice);
213             String estadisticasMeta = metaEscogida.porcentajeCumplimiento(indice);
214
215             if (verificadorMeta){
216
217                 System.out.println("\nLa meta ha sido cumplida exitosamente");
218                 System.out.println("Sumaremos al pago la bonificación por esta meta");
219                 valorPorMetas += metaEscogida.getComision();
220                 trabajadorEscogido.getCumplimientoMetas().set(opcMetaEscogida - 1,true);
221
222             } else{
223
224                 System.out.println("\nLa meta aún no ha sido cumplida");
225
226             }
227
228             System.out.println(estadisticasMeta);

```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

- Se selecciona la meta. En este caso la 1

```
> 1

El trabajador escogido tiene las siguientes metas

Indice: número de productos vendidos

Meta 1

Minimo requerido:          6.0
Bonificación por cumplimiento: 12000.0

Meta 2

Minimo requerido:          12.0
Bonificación por cumplimiento: 25000.0

Seleccione una de las metas
```

- Esta meta ya fue cumplida, por lo que se suma el valor de la comisión al pago del trabajador y se pregunta si se quiere revisar otra meta

```
> 1

La meta ha sido cumplida exitosamente
Sumaremos al pago la bonificación por esta meta
Porcentaje de cumplimiento: 100.0%

¿Qué desea hacer?
1.Revisar otra meta
2.Proceder con el pago
```

- Podemos revisar otra meta seleccionando la opción 1

```

-----
> 1

Seleccione una de las metas
> 2

La meta aún no ha sido cumplida
Porcentaje de cumplimiento: 50.0%
Porcentaje faltante: 50.0%
Cantidad faltante del índice indicado: 6.0

¿Qué desea hacer?
1.Revisar otra meta
2.Proceder con el pago

```

- Seleccionamos la meta 2, pero como esta no ha sido cumplida, se deja el valor del pago como estaba. Ahora, seleccionaremos la opción de proceder con el pago

```

> 2
... Realizando pago ... Por favor espere ...

El pago fue realizado con éxito

-----
Comprobante de pago
Pago asociado a los envios realizados 350000
Pago asociado al cumplimiento de metas 12000
Total = 362000
-----

```

Por otro lado, si el usuario selecciona que no desea bonificar al usuario, se procede a hacer el pago con el valor del pago básico obtenido anteriormente.

Esto se puede ver en la consola al ejecutar el código de la siguiente manera:

```

> 2
... Realizando pago ... Por favor espere ...

El pago fue realizado con éxito

-----
Comprobante de pago
Pago asociado a los envios realizados 70000
Pago asociado al cumplimiento de metas 0
Total = 70000
-----

¿Qué desea hacer?
1.Pagar a otro trabajador
0.Volver al menu principal
>

```

Ahora, para realizar al pago al trabajador, se suma el pago básico y las bonificaciones por metas cumplidas. Para enviar el dinero, se usa el método *recibirPagos()* de la clase del trabajador seleccionado (Método abstracto implementado por las clases **Operario**, **Vendedor** y **Transportador**)

Luego, se muestra un mensaje de progreso con tiempos de espera simulados para hacer más realista el proceso, para después mostrar un comprobante de pago detallando el monto total pagado.

Esta parte se puede observar en código de la siguiente manera:

```

269         trabajadorEscogido.recibirPagos(total + valorPorMetas);
270
271
272         // temporizador para que se vea mas real la eliminacion.
273         try {
274
275             Thread.sleep(500); // Espera 1 segundo
276
277         } catch (InterruptedException e) {
278
279             e.printStackTrace();
280         }
281
282         System.out.println("... Realizando pago ... Por favor espere ...");
283
284         try {
285
286             Thread.sleep(1000); // Espera 1,5 segundos
287
288         } catch (InterruptedException e) {
289
290             e.printStackTrace();
291         }
292
293
294         System.out.println("\nEl pago fue realizado con éxito\n");
295         System.out.println("-----");
296         System.out.println("Comprobante de pago");
297         System.out.println("Pago asociado a los envios realizados " + total);
298         System.out.println("Pago asociado al cumplimiento de metas " + valorPorMetas);
299         System.out.println("Total = " + (total + valorPorMetas));
300         System.out.println("-----");

```

Luego, el contador de trabajo del trabajador se resetea a 0 usando el método `getTrabajado()` de la clase **Persona** (Definido en la línea 190)

Esta parte se puede observar en código de la siguiente manera:

```
304 | trabajadorEscogido.setTrabajado(0);
```

Por último, se le pregunta al usuario si desea pagar a otro trabajador o volver al menú principal. Dependiendo de la selección del usuario, se reinicia el ciclo principal o se termina la funcionalidad.

Esta parte se puede observar en código de la siguiente manera:

```
314 | // Ciclo final para ver si sale o se reinicia la funcionalidad  
315 | System.out.println("\n¿Qué desea hacer?\n1.Pagar a otro trabajador\n0.Volver al menu principal");  
316 |  
317 | while (true) {  
318 |  
319 |     System.out.print("> ");  
320 | opcSalida = sc.nextInt();  
321 |  
322 |     if (opcSalida == 1) {  
323 |  
324 |         verificador1 = true;  
325 |         break;  
326 |  
327 |     } else if (opcSalida == 0) {  
328 |  
329 |         verificador1 = false;  
330 |         break;  
331 |  
332 |     } else {  
333 |  
334 |         System.out.println("Seleccione una de las opciones disponibles: ");  
335 |     }  
336 | }
```

Adicionalmente, esto se puede ver en la consola al ejecutar el código de la siguiente manera:

```
¿Qué desea hacer?  
1.Pagar a otro trabajador  
0.Volver al menu principal
```

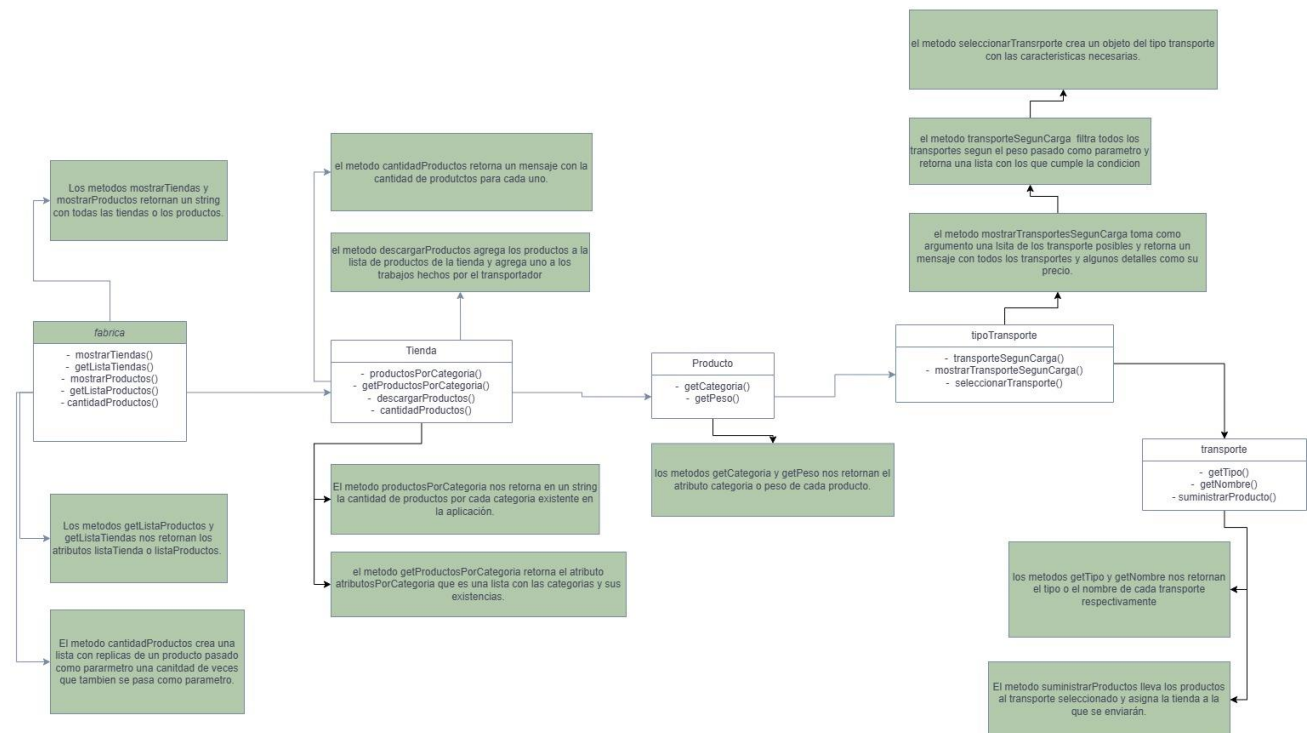
❖ Funcionalidad 3: Proveer tiendas

Esta funcionalidad cumple el objetivo de poder agregar existencias de cualquier producto a las tiendas, gestionando el suministro de productos.

Interacciones: **Fabrica** envía **Producto** a **Tienda** usando un **Transporte**. De la siguiente manera:

Link: <https://drive.google.com/file/d/1HiQDy9xitWesD5zBazYMeaqdcuC1XJmH/view>

Proveer Tienda



La funcionalidad está construida en una serie de pasos para poder agregar las existencias deseadas de manera correcta.

1. Selección de tienda:

El primer paso es que el usuario seleccione la tienda, esto se logra mostrando todas las tiendas disponibles con el método `mostrarTiendas()` de la clase **Fabrica** (Definido en la línea 140).

El usuario ingresa el valor que corresponde a la tienda escogida. Luego, se verifican las restricciones correspondientes a este valor, haciendo que si este es igual a cero se regresa al menú principal, si el valor está entre uno y el máximo de tiendas seguimos con el proceso (El valor máximo lo obtenemos gracias al método *getListaTienda()* de la clase **Fabrica** (Definido en la línea 313) que nos retorna en una lista todas las tiendas y tomamos el tamaño de la lista como el máximo número de tiendas), y en caso de que el número ingresado por el usuario no esté en el rango permitido se le informa al usuario que el número no es válido y se le pide que lo reingrese.

Cuando validamos la opción del usuario guardamos la tienda escogida en el atributo *tiendaSeleccionado*, usando el valor ingresado menos 1 para acceder a la tienda seleccionada.

Esta primera parte se puede observar en el código de la siguiente manera:

```
63         case 1:
64
65             // Visto en pantalla
66             System.out.println("\n");
67             System.out.println("Proveer tiendas - Apartado de tiendas");
68             System.out.println("\n0. Volver al menu anterior\n");
69             System.out.print(Cargar.fabrica.mostrarTiendas());
70             // Seleccionar tienda
71             System.out.print("Seleccione la tienda a la que desea enviar: ");
72             // Entero seleccionado
73             x = escaner1.nextInt();
74
75
76             while (interruptor) {
77                 // Se establece el intervalo en el que estan las tiendas
78                 if (x == 0) {
79                     eleccion = 0;
80                     break;
81                 }
82
83                 if (x > 0 && x <= Cargar.fabrica.getListaTienda().size()) {
84                     tiendaSeleccionada = Cargar.fabrica.getListaTienda().get(x - 1);
85                     eleccion = 2;
86                     break;
87                 } else if (x != 0) {
88                     System.out.print("Por favor seleccione una tienda dentro del rango: ");
89                     x = escaner1.nextInt();
90                 }
91             }
92
93
94
95
96     }
```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

Proveer tiendas - Apartado de tiendas

0. Volver al menu anterior

1. Delicia Fresca Norte-Productos:

Zanahoria: 4
Pepino: 4
Limon: 4
Piña: 4
Pera: 4
Pan: 4
Sanduche: 4
Pandebono: 4
Pan hojaldrado: 4
Galletas: 4
Palito de queso: 4
Mermelada de frambuesa: 4
Mermelada de piña: 4
Mermelada de mora: 4
Mayonesa: 4
Salsa de tomate: 4
Mermelada de fresa: 4
Te verde: 4
Jugo de naranja: 4
Leche de almendras: 4
Leche de avena: 4
Yogur de soya: 4
Cafe instantaneo: 4
Jugo de mora: 4
Manzana: 3

2. Delicia Fresca Sur-Productos:

Pera: 4
Tomate: 4
Frambuesa: 4
Lechuga: 4
Sandia: 4
Pan: 4
Palito de queso: 4
Tostada: 4
Ponque: 4
Pan tajado: 4
Almojabana: 4
Mermelada de frambuesa: 4
Mermelada de fresa: 4
Mermelada de kiwi: 4
Mermelada de frutos rojos: 4
Salsa BBQ: 4
Salsa de soya: 4
Te verde: 4
Cafe instantaneo: 4
Bebida achocolatada: 4
Leche de coco: 4
Bebida hidratante: 4
Jugo de mora: 4
Pepino: 4
Jugo de naranja: 4

3. Delicia Fresca Centro-Productos:

Manzana: 4
Zanahoria: 4
Limon: 4
Piña: 4
Tomate: 4
Frambuesa: 4
Lechuga: 4
Sandia: 4
Sanduche: 4
Pandebono: 4
Pan hojaldrado: 4
Galletas: 4
Tostada: 4
Ponque: 4
Pan tajado: 4
Almojabana: 4
Mermelada de piña: 4
Mermelada de mora: 4
Mayonesa: 4
Salsa de tomate: 4
Mermelada de kiwi: 4
Mermelada de frutos rojos: 4
Salsa BBQ: 4
Salsa de soya: 4
Leche de almendras: 4
Leche de avena: 4
Yogur de soya: 4
Bebida achocolatada: 4
Leche de coco: 4
Bebida hidratante: 4

Seleccione la tienda a la que desea enviar:

2. Selección de producto

El siguiente paso es elegir el producto a enviar, primero se muestra la capacidad de productos de la tienda con el método de la clase *productosPorCategoria()* de la clase **Tienda** (Definido en la línea 266). Luego, se muestran todas las opciones de productos disponibles para elegir usando el método *mostrarProductos()* de la clase **Fabrica** (Definido en la línea 115).

Posteriormente el usuario deberá elegir una opción que será validada teniendo en cuenta que si el valor escogido esta entre 1 y el máximo de productos, se guardará el valor en el atributo *productoSeleccionado*, si no cumple esta condición se le informa al usuario que el número no es válido y se le pide que lo reingrese (El valor máximo lo obtenemos gracias al método *getListaProductos()* de la clase **Fabrica** (Definido en la línea 301) que nos retorna en una lista todos los productos y tomamos el tamaño de la lista como el máximo número de productos).

Esta parte se puede observar en el código de la siguiente manera:

```
101         case 2:
102
103             System.out.println("\nProveer tiendas - Apartado de productos");
104             System.out.print("\nLa capacidad de productos por categoria para esta tienda es la siguiente: \n");
105             System.out.println(tiendaSeleccionada.productosPorCategoria());
106             System.out.println("\n0. Regresar al menu anterior");
107             System.out.println(Cargar.fabrica.mostrarProductos());
108             System.out.print("Seleccione el producto que desea enviar: ");
109
110             while (x != 0) {
111                 escanerInt = escaner2.nextInt();
112                 // Se establece el intervalo en el que estan los productos
113
114                 if (escanerInt == 0) {
115                     eleccion = 1;
116                     break;
117                 }
118
119                 if (escanerInt > 0 && escanerInt <= Cargar.fabrica.getListaProductos().size()) {
120
121                     productoSeleccionado = Cargar.fabrica.getListaProductos().get(escanerInt - 1);
122                     eleccion = 3;
123                     break;
124                 } else {
125
126                     System.out.print("Por favor seleccione un producto dentro del rango: ");
127                 }
128             }
129
130         }
131
132         break;
```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```
Proveer tiendas - Apartado de productos
```

```
La capacidad de productos por categoria para esta tienda es la siguiente:  
frutas y verduras 23/81 panaderia 24/56 salsas y mermeladas 24/82 bebidas 28/64
```

```
0. Regresar al menu anterior
```

```
NÚMERO-PRODUCTO-PESO-PRECIO-CATEGORIA
```

```
1. Manzana - 2.5 - 3000.0 - frutas y verduras  
2. Zanahoria - 2.0 - 2000.0 - frutas y verduras  
3. Pepino - 0.5 - 500.0 - frutas y verduras  
4. Limon - 1.0 - 800.0 - frutas y verduras  
5. Piña - 3.0 - 3500.0 - frutas y verduras  
6. Pera - 2.5 - 4000.0 - frutas y verduras  
7. Tomate - 2.0 - 1500.0 - frutas y verduras  
8. Frambuesa - 1.0 - 6000.0 - frutas y verduras
```

Aquí se muestran el resto de los productos disponibles, y por último se muestra el enunciado:

```
Seleccione el producto que desea enviar:
```

3. Ingresar cantidad

Como siguiente paso se le preguntará al usuario que cantidad del producto seleccionado desea ingresar. Para validar la opción ingresada por el usuario obtenemos la categoría del producto ingresado con el método *getCategoria()* de la clase **Producto** (Definido en la línea 229) que nos retorna un string con la categoría del producto; luego de esto usamos el método *getProductosPorCategoria()* de la clase **Tienda** (Definido en la línea 457) que nos retorna una lista con objetos de tipo **Pareja** que contienen como clave las categorías de los productos y como valor la cantidad de productos de esta categoría en la tienda seleccionada, en esta lista buscaremos la categoría del producto seleccionado y guardamos el valor obtenido en la el atributo *productoEnTiendaPorCategoria*.

Posteriormente, se repite el mismo proceso usando el método *getCantidadPorCategoria()* de la clase **Tienda** (Definido en la línea 445) y este nos retorna las categorías con el máximo de productos permitidos para esta categoría, el cual guardaremos en el atributo *productosMaximosEnTiendaPorCategoria*.

Teniendo en cuenta estos valores, diremos que *productosMaximosEnTiendaPorCategoria* será el valor máximo de productos permitidos de esta categoría y *productoEnTiendaPorCategoria* será la cantidad que hay de productos de esta categoría en ese momento en la tienda, por lo que la cantidad de productos ingresados por el usuario deberá ser igual o menor a la diferencia de estos valores

(*productosMaximosEnTiendaPorCategoria* - *productosMaximosEnTiendaPorCategoria*), si no es así, se pedirá al usuario ingresar otro valor dentro del rango.

Esta parte se puede observar en el código de la siguiente manera:

```
135         case 3: // Cantidad de productos
136
137             System.out.print("\nEscriba la cantidad de productos que desea proveer: ");
138             int productoEnTiendaPorCategoria = 0;
139             int productosMaximosEnTiendaPorCategoria = 0;
140             String categoriaProducto = productoSeleccionado.getCategoria(); // Asumiendo que getCategoria() devuelve un String
141
142             // Buscar en productosPorCategoria
143             for (Parejas<String, Integer> par : tiendaSeleccionada.getProductosPorCategoria()) {
144
145                 if (par.getKey().equals(categoriaProducto)) {
146                     productoEnTiendaPorCategoria = par.getValue();
147                     break;
148                 }
149             }
150
151             // Buscar en cantidadPorCategoria
152             for (Parejas<String, Integer> par : tiendaSeleccionada.getCantidadPorCategoria()) {
153
154                 if (par.getKey().equals(categoriaProducto)) {
155                     productosMaximosEnTiendaPorCategoria = par.getValue();
156                     break;
157                 }
158             }
159
160
161             while (true) {
162
163                 escanerInt = escaner2.nextInt();
164
165                 if (escanerInt == 0) {
166                     eleccion = 1;
167                     break;
168                 }
169
170                 //Se hace con el fin de evitar que intente mandar mas productos de los que soporta la tienda por la respectiva categoria
171                 else if (escanerInt < 0 || escanerInt >= productosMaximosEnTiendaPorCategoria - productoEnTiendaPorCategoria) {
172
173                     eleccion = 4;
174                     break;
175                 } else {
176
177                     System.out.print("Por favor seleccione una cantidad en el limite de la tienda por categoria: ");
178                 }
179             }
180
181             break;
182
```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```
Escriba la cantidad de productos que desea proveer:
```

En caso de que el numero ingresado no esté en el rango indicado, se mostrará el siguiente mensaje:

```
Escriba la cantidad de productos que desea proveer: 100
Por favor seleccione una cantidad en el limite de la tienda por categoria:
```

4. Seleccionar transporte:

Luego de obtener la cantidad del producto, le solicitamos al usuario el tipo de transporte para realizar el envío del producto. Para seleccionar las opciones posibles, primero obtenemos el peso total del producto multiplicando la cantidad de producto por el peso de una unidad (se obtiene con el método *getPeso()* de la clase **Producto** (Definido en la línea 193). Con el peso calculado filtramos los transportes posibles con el método *transporteSegunCarga()* de la clase **TipoTransporte** (Definido en la línea 84) y guardamos la lista que retorna en el atributo *listaTransFiltrada()*, que será enviada como parámetro al método *mostrarTransporteSegunCarga()* de la clase **TipoTransporte** (Definido en la línea 110) que le mostrará al usuario las opciones de transporte posibles.

En este punto, se verifica que la opción ingresada por el usuario sea mayor a 0 y menor a la longitud de la lista de los transportes posibles, si no cumple con esto, se le pedirá ingresar otro valor dentro del rango. Si el valor resulta valido, se guardará en el atributo *transporteSeleccionado* usando el método *seleccionarTransporte()* de la clase **TipoTransporte** (Definido en la línea 136).

Esta parte se puede observar en el código de la siguiente manera:

```
185         case 4: // seleccionar tipo de transporte
186
187             int PesoTotalProductos = escanerInt * ((int) Math.round(productoSeleccionado.getPeso()));
188             System.out.println("\n\nSeleccione en que medio de transporte quiere enviar este producto");
189             System.out.println(
190                 "\nAdvertencia: Los tipos de transporte han sido filtrados de manera que solo puede "
191                 + "seleccionar los que puedan soportar el peso de su producto.");
192
193             System.out.println("0. Regresar al menu anterior");
194
195             // TipoTransporte tipoTransportes;
196             ArrayList<TipoTransporte> listaTransFiltrada = new ArrayList<TipoTransporte>();
197             listaTransFiltrada = TipoTransporte.transporteSegunCarga(PesoTotalProductos);
198
199             System.out.println(TipoTransporte.mostrarTransporteSegunCarga(listaTransFiltrada));
200             System.out.println("Seleccione el número del tipo de transporte: ");
201             System.out.print("> ");
```

```

202         while(true){
203
204             int numTransporteSeleccionado = escaner2.nextInt();
205
206             if (numTransporteSeleccionado == 0) {
207
208                 eleccion = 2;
209                 break;
210             }
211
212             if (numTransporteSeleccionado > listaTransFiltrada.size() || numTransporteSeleccionado < 0) {
213
214                 System.out
215                     .print("Número de transporte inválido, por favor seleccione un producto en la lista \n> ");
216             }
217
218             else {
219
220                 transporteSeleccionado = TipoTransporte.seleccionarTransporte(listaTransFiltrada,
221                                         numTransporteSeleccionado);
222                 System.out.print("Ha seleccionado el transporte #" + (numTransporteSeleccionado)
223                                 + "\nLa tienda se proveerá por: " + transporteSeleccionado.getTipo().getNombre());
224                 eleccion = 5;
225                 break;
226             }}
227
228         break;
229

```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```

Seleccione en que medio de transporte quiere enviar este producto

Advertencia: Los tipos de transporte han sido filtrados de manera que solo puede seleccionar los que puedan soportar el peso de su producto.
0. Regresar al menu anterior
1. Caminando $1000.0
2. Bicicleta $2000.0
3. Moto $5000.0
4. Carro $10000.0
5. Camion $50000.0
6. Avion $500000.0

Seleccione el número del tipo de transporte:

```

5. Envío de los productos:

Posterior a elegir el medio de transporte se hace el envío de los productos a la tienda, para esto primero descontamos los productos de la fábrica con el método *cantidadProductos()* de la clase **Fabrica** (Definido en la línea 233) y estos productos son cargados al transporte escogido. Después de esto usamos el método *suministrarProductos()* de la clase **Transporte** (Definido en la línea 94) para enviar los productos a la tienda seleccionada.

Luego, verificamos que la tienda si se haya guardado correctamente, viendo si el transporte tiene en su atributo *tienda* la que corresponde a la tienda que se había escogido previamente. Esto lo podemos ver con el método *getTienda()* de la clase **Transporte** (Definido en la línea 262).

Después de hacer esta verificación, usamos el método *descargarProducto()* de la clase **Tienda** (Definido en la línea 247) para guardar las nuevas existencias del producto en la

tienda seleccionada. Si la entrega fue realizada con éxito se le muestra la nueva cantidad de productos al usuario usando el atributo *cantidadProductos* de la clase **Tienda**, si no se logra se le informa al usuario que el envío no pudo ser realizado.

Esta parte se puede observar en el código de la siguiente manera:

```
231         case 5:
232
233             listaDeProductos = Cargar.fabrica.cantidadProductos(escanerInt, productoSeleccionado); // meter los productos en el camion
234             transporteSeleccionado.suministrarProducto(tiendaSeleccionada, listaDeProductos); // Se envían los productos en la tienda,
235
236             if (transporteSeleccionado.getTienda().equals(tiendaSeleccionada) == true) {
237
238                 tiendaSeleccionada.descargarProducto(transporteSeleccionado);
239                 System.out
240                     .println("\nEl producto fue enviado con éxito ahora la tienda tiene \nPRODUCTOS: "
241                         + tiendaSeleccionada.cantidadProductos());
242
243             } else {
244
245                 System.out.println("El envío no se pudo realizar a esa tienda");
246             }
247
```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```
Ha seleccionado el transporte #1
La tienda se proveerá por: Caminando
El producto fue enviado con éxito ahora la tienda tiene
PRODUCTOS:
Zanahoria: 4
Pepino: 4
Limon: 4
Piña: 4
Pera: 4
Pan: 4
Sanduche: 4
Pandebono: 4
Pan hojaldrado: 4
Galletas: 4
Palito de queso: 4
Mermelada de frambuesa: 4
Mermelada de piña: 4
Mermelada de mora: 4
Mayonesa: 4
Salsa de tomate: 4
Mermelada de fresa: 4
Te verde: 4
Jugo de naranja: 4
Leche de almendras: 4
Leche de avena: 4
Yogur de soya: 4
Cafe instantaneo: 4
Jugo de mora: 4
Manzana: 4
```

Por último, se le pregunta al usuario si desea hacer el mismo proceso para otra tienda o producto, o si desea volver al menú principal.

Esta parte se puede observar en el código de la siguiente manera:

```
248 // Ciclo final para ver si sale o se reinicia la funcionalidad
249 System.out.println("\n0.Volver al menu principal" + "\n" + "1. Proveer más tiendas");
250 System.out.print("> ");
251
252 while (true) {
253
254     escanerInt = escaner2.nextInt();
255
256     if (escanerInt == 0) {
257
258         eleccion = 0;
259         break;
260
261     } else if (escanerInt == 1) {
262
263         eleccion = 1;
264         break;
265
266     } else {
267
268         System.out.println("Seleccione una de las opciones disponibles: ");
269     }
270 }
271 break;
272
273
274
275
276 }
```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

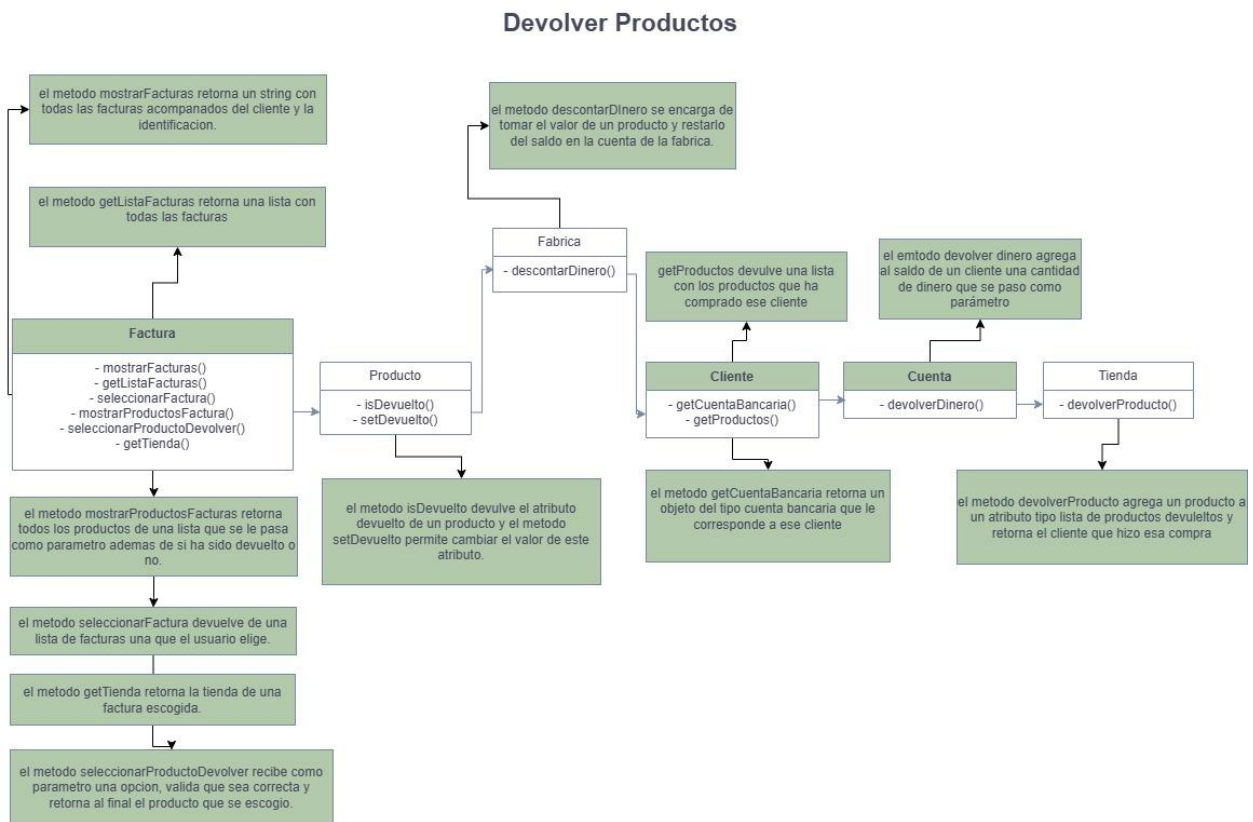
```
0.Volver al menu principal
1. Proveer más tiendas
```


❖ Funcionalidad 4: Devoluciones de productos

Permite la devolución de productos por parte de un cliente, actualizando el inventario y reembolsando el dinero.

Interacciones: Cliente devuelve Producto a Tienda, se actualiza la Factura y la CuentaBancaria. De la siguiente manera:

Link: <https://drive.google.com/file/d/14ev2OM564qgJrk9UpETIfaLbDwaSmABT/view>



Esta funcionalidad maneja distintas opciones para verificar las condiciones a cumplir para que el producto pueda ser devuelto y al mismo tiempo en estas opciones se lleva un paso a paso para hacer la correcta devolución del producto.

El primer paso en este método es ver las facturas y que el usuario elija la factura en donde está el producto que quiere ser devuelto. Para esto, usamos el método *mostrarFacturas()* de la clase **Factura** (Definido en la línea 427) que retorna un string con la identificación de la factura y el nombre del cliente al que corresponde. Este retorno es guardado en el atributo *facturas*, que será impreso para que el usuario pueda seleccionar la factura de la que desea hacer la devolución. Aquí se verificará que la opción ingresada este entre 1 y el máximo de facturas para poder continuar con el proceso (El máximo de facturas se obtiene de llamar al método *getListaFacturas()* de la clase **Factura** (Definido en la línea 708) el cual devuelve una lista con todas las facturas y tomamos el tamaño de este arreglo para obtener el máximo)

Luego de hacer la verificación, se guarda la factura en la variable *facturaSeleccionada* con el método *seleccionarFactura()* de la clase **Factura** (Definida en la línea 454).

Esta parte se puede observar en el código de la siguiente manera:

```

56         case 1: //Seleccionar factura
57
58             while (condicion1) {
59                 String facturas = Factura.mostrarFacturas(); // se almacenan las facturas guardadas en la clase Factura dentro de un String
60                 System.out.println("\nPor favor seleccione el número que le corresponda\n" + "a la factura para realizar la devolución\n");
61                 System.out.println("0. Volver al menu principal");
62                 System.out.println(facturas); // se muestran las facturas en pantalla
63                 System.out.println("Digite su opcion: ");
64                 System.out.print("-----");
65                 int op = sc.nextInt(); // el usuario ingresa la opción seleccionada
66
67                 if (op == 0) { // Volver al menú principal
68
69                     eleccion = 0;
70                     condicion1 = false;
71                     condicion2 = false;
72                     condicion3 = false;
73                     break;
74
75                 } else if ((op > 0) && (op <= Factura.getListaFacturas().size())) {
76
77                     facturaSeleccionada = Factura.seleccionarFactura(op);
78                     System.out.println("Seleccionó la factura con la opcion número: " + op);
79                     eleccion = 2;
80                     break;
81
82                 } else {
83
84                     System.out.println("El número que ha ingresado no esta entre las opciones disponibles");
85
86                 }
87             }
88

```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```

Por favor seleccione el número que le corresponda
a la factura para realizar la devolución

0. Volver al menu principal
1. Cliente: Esteban Carrillo Identificación de compra

Digite su opcion:

```

El siguiente paso se encarga de verificar que en la factura seleccionada existan productos para devolver, esto se hace creando un ciclo en donde con el método *getListaProductos()* de la clase **Factura** (Definido en la línea 696) obtenemos todos los productos de la factura seleccionada y posteriormente para cada uno de estos verificamos con el método *isDevuelto()* de la clase **Producto** (Definido en la línea 241) si el producto ya ha sido devuelto, en caso afirmativo sumamos uno al atributo usado como contador *contadorDevoluciones*. Cuando se termina el proceso con el último producto verificamos con un condicional si el valor en *contadorDevoluciones* es mayor o igual a la cantidad de los productos de la factura, si la condición se cumple le pedimos al usuario que seleccione otra factura.

Esta parte se puede observar en el código de la siguiente manera:

```

90         case 2:
91
92             while (condicion2) {
93
94                 // En este ciclo se revisa si en la factura ya estan todos los productos devueltos
95                 // Si estan todos devueltos se le informa al usuario y se le pide ingresar el número de otra factura
96
97                 int contadorDevoluciones = 0; // Conteo de los productos que ya han sido devueltos
98
99                 for (Producto producto : facturaSeleccionada.getListaProductos()) {
100
101                     if (producto.isDevuelto()) {
102                         contadorDevoluciones++;
103                     }
104                 }
105
106                 if (contadorDevoluciones >= facturaSeleccionada.getListaProductos().size()) {
107                     System.out.println("En la factura que seleccionó ya se han devuelto todos los productos\n" +
108                                     "Por favor digite el Número de otra factura\n");
109
110                     eleccion = 1; // Se vuelve al caso 1 para muestre las facturas de nuevo
111                     break;
112                 }

```

En caso de verificar que en la factura seleccionada se puede devolver algún producto se inicia el último paso de esta funcionalidad en el cual haremos la devolución del producto. Primero, al método *mostrarProductosFactura()* de la clase **Factura** (Definido en la línea 472) le pasamos como parámetro los valores devueltos del método *getListaProductos()* de la clase **Factura** (Definido en la línea 696), con esto se obtienen todos los productos de la factura para almacenarlos en el atributo *productosFactura*, que será impresos para que el usuario pueda seleccionar el producto que desea devolver.

Esto lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```

Seleccionó la factura con la opcion número: 1

Por favor seleccione el número que le corresponda
a el producto para realizar la devolución

0. Volver al menu principal
1. Producto: Manzana

Dígame su opcion:

```

Si la opción del usuario se encuentra entre 1 y el máximo de productos en la factura (El máximo de productos se obtiene de tomar el tamaño de la lista que nos devuelve el método *getListaProductos()* de la clase **Factura** (Definido en la línea 696)). Se verifica además que el producto escogido no haya sido devuelto con el método *isDevuelto()* de la clase **Producto** (Definido en la línea 241).

En caso de que ambas condiciones se hayan cumplido se completan los últimos pasos para devolver el producto. Primero usamos el método *setDevuelto()* de la clase **Producto** (Definido en la línea 247) para establecer el atributo *devuelto* del producto a *True* y evitar que se pueda volver a devolver.

Esta parte se puede observar en el código de la siguiente manera:

```

114 String productosFactura = Factura.mostrarProductosFactura(facturaSeleccionada.getListaProductos()); // Se guardan los productos de la factura
115
116 System.out.println("\nPor favor seleccione el número que le corresponda\n" + "a el producto para realizar la devolución\n");
117 System.out.println("0. Volver al menu principal");
118 System.out.println(productosFactura); // se muestran los productos de la factura
119 System.out.println("Dígame su opcion: ");
120 System.out.print("> ");
121 int op2 = sc.nextInt(); // el usuario ingresa la opción seleccionada
122 System.out.println("");
123
124 productoSeleccionado = facturaSeleccionada.seleccionarProductoDevolver(op2);
125
126 if (op2 == 0) {
127
128     eleccion = 0;
129     condicion2 = false;
130     condicion3 = false;
131     break;
132 }
133
134 else if ((op2 > 0) && (op2 <= facturaSeleccionada.getListaProductos().size()) && (!productoSeleccionado.isDevuelto())) {
135
136     Boolean devolver = true;
137     productoSeleccionado.setDevuelto(devolver);
138     System.out.println("Selección el producto con la opcion número: " + op2);

```

Después de esto usamos el método *descontarDinero()* de la clase **Fabrica** (Definido en la línea 185) pasándole como parámetro el producto seleccionado y de este modo descontamos el dinero del producto a la tienda que lo vende. Luego, accedemos a la tienda correspondiente a la factura usando el método *getTienda()* de la clase **Factura** (Definido en la línea 624), para luego usar el método *devolverProducto()* de la clase **Tienda** (Definido en la línea 356)

para agregar el producto seleccionado a un atributo de tipo ArrayList llamado *productosDevueltos*.

Por último, se obtiene la cuenta Bancaria del cliente que compró el producto con el método *getCuentaBancaria()* de la clase **Cliente** (Definido en la línea 163). Luego, en esta cuenta bancaria se usa el método *devolverDinero()* de la clase **cuentaBancaria** (Definido en la línea 135) y se le envía como parámetros la cantidad de dinero que cuesta el producto devuelto y el nombre del cliente.

Al mismo tiempo, se obtienen los productos del cliente con el método *getProductos()* de la clase **Cliente** (Definido en la línea 175) y se remueve el producto devuelto de la lista que nos devuelve este método.

Esta parte se puede observar en el código de la siguiente manera:

```
150         System.out.println("Se esta realizando devolución, por favor espere");
151
152         try {
153
154             Thread.sleep(1000); // Espera 1 segundo
155
156         } catch (InterruptedException e) {
157
158             e.printStackTrace();
159         }
160
161         double total = Cargar.fabrica.descontarDinero(productoSeleccionado);
162
163         Cliente cliente = facturaSeleccionada.getTienda().devolverProducto(facturaSeleccionada, productoSeleccionado);
164
165         cliente.getCuentaBancaria().devolverDinero(total, cliente);
166
167         cliente.getProductos().remove(productoSeleccionado); // se elimina el producto que compró el cliente
168
169         System.out.println("El producto ha sido devuelto");
170
171         try {
172
173             Thread.sleep(750);
174
175         } catch (InterruptedException e) {
176
177             e.printStackTrace();
178         }
179
180         eleccion = 3; // para que entre al ultimo caso
181         break;
```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```
Seleccionó el producto con la opcion número: 1
Se esta realizando devolución, por favor espere
El producto ha sido devuelto
```

Por último, se muestra un mensaje de confirmación de que el producto ha sido devuelto y se le pregunta al usuario si desea volver al menú principal o si desea hacer otra devolución.

Esta parte se puede observar en el código de la siguiente manera:

```
190
191     case 3:
192
193         while (condicion3) {
194
195             System.out.println("¿Desea hacer otra devolucion o volver al menu principal?\n");
196             System.out.println("0. Volver al menu principal");
197             System.out.println("1. Realizar otra devolución");
198             System.out.print("> ");
199
200             int opcion = sc2.nextInt();
201
202             if (opcion == 0) {
203
204                 eleccion = 0;
205                 condicion = false;
206                 break;
207
208             } else if (opcion == 1) {
209
210                 eleccion = 1;
211                 break;
212
213             } else {
214                 System.out.println("La opcion elegida no es válida ");
215             }
216         }
217         break;
218
219     }
```

Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```
El producto ha sido devuelto
¿Desea hacer otra devolucion o volver al menu principal?

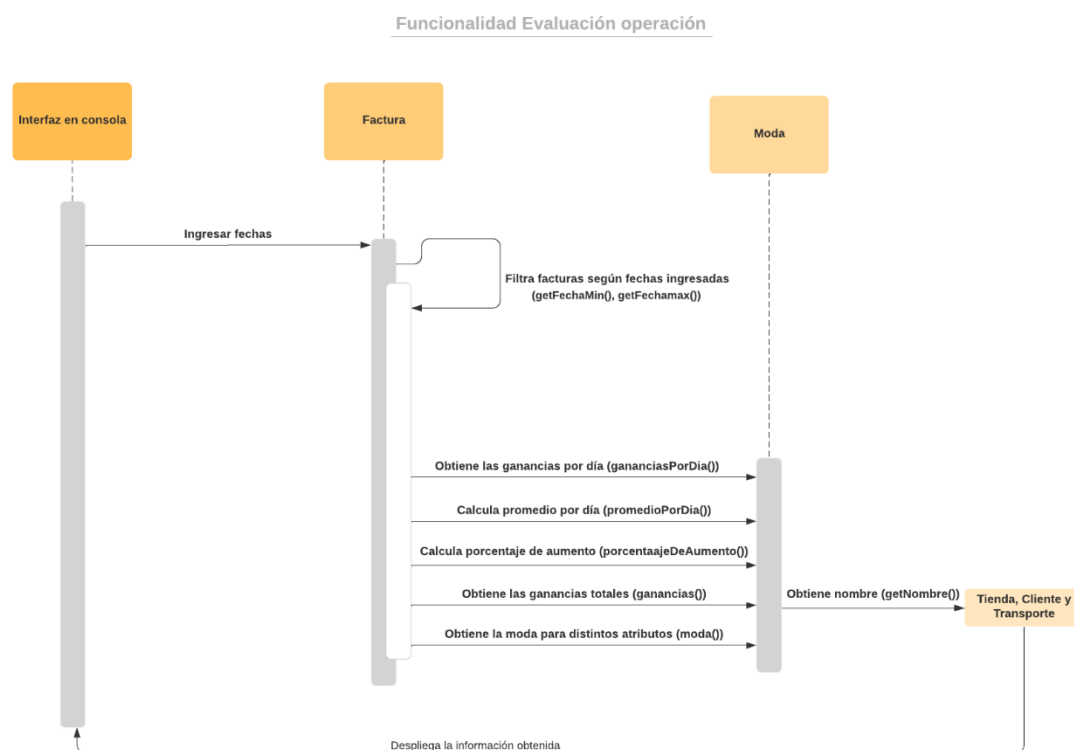
0. Volver al menu principal
1. Realizar otra devolución
>
```

❖ Funcionalidad 5: Evaluación operación

Esta funcionalidad permite evaluar el desempeño de las operaciones de la empresa, utilizando la información que se almacena cada vez que se realiza un pedido. La evaluación incluye el análisis de ganancias diarias, totales, promedio por día y porcentaje de aumento, así como la identificación de las tiendas, clientes y transportes más utilizados.

Interacciones: En esta funcionalidad se interactúa con objetos de tipo **Factura**, tipo **Moda**, tipo **Tienda**, tipo **Cliente** y tipo **Transporte**. De la siguiente manera:

Link: <https://lucid.app/lucidchart/38591d0e-6534-4a48-a263-2eda5a976d27/view>



La información acerca de cada uno de los pedidos realizados se guarda en el atributo *listaFacturas* de la clase **Factura**, teniendo toda la información de cada Factura. Adicionalmente, como cada **Factura** tiene un atributo *fecha*, se puede tener un orden cronológico para el manejo de estas facturas.

Esta funcionalidad se divide en varias secciones:

1. Ingreso de fechas para análisis:

El primer paso es especificar el rango de fechas para el análisis utilizando el método *ingresarFechas()* de la clase **EvaluacionOperacion** (Definido en la línea 53). Este método permite al usuario ingresar fechas específicas o analizar toda la información disponible.

Esta parte se puede observar en el código de la siguiente manera:

```
53 public static int[] ingresarFechas(){
54
55     Scanner sc = new Scanner(System.in);
56     int opcion = new MenuAuxiliar("REPORTE", new String[]{"Analizar toda la información",
57                                                         "Ingresar fechas específicas"}).mostrar();
58
59     int fechaMin = Factura.getFechaMin();
60     int fechaMax = Factura.getFechaMax();
61
62     while(opcion != 0){
63
64         // Manejo de opciones
65         switch(opcion){
66
67             case 1:
68
69                 return new int[]{fechaMin, fechaMax};
70
71
72             case 2:
73
74                 System.out.println("\nLa fecha mínima es " + fechaMin + " y la fecha máxima es " + fechaMax);
75
76                 System.out.println("Ingrese fecha de inicio: ");
77                 int fechal = MenuAuxiliar.ingresarConLimites(fechaMin, fechaMax);
78
79                 System.out.println("Ingrese fecha final: ");
80                 int fecha2 = MenuAuxiliar.ingresarConLimites(fechal, fechaMax);
81
82                 return new int[]{fechal, fecha2};
83
84
85             case 0:
86
87                 return new int[]{-1, -1};
88
89             default:
90
91                 System.out.println("Ha ingresado un valor no válido. Por favor vuelva a intentarlo");
92
93                 break;
94
95         }
96
97         opcion = new MenuAuxiliar("REPORTE", new String[]{"Analizar toda la información",
98                                                         "Ingresar fechas específicas"}).mostrar();
99     }
100
101     return new int[]{-1, -1};
102
103 }
```


Adicionalmente, lo podemos ver en la consola al ejecutar el código de la siguiente manera:

```
REPORTE

1. Analizar toda la información
2. Ingresar fechas específicas
0. Cancelar
```

Se puede elegir analizar toda la información o ingresar un periodo en específico. Si se decide ingresar fechas, por consola se mostrará lo siguiente:

```
La fecha mínima es 1 y la fecha máxima es 1
Ingrese fecha de inicio:
```

(Acá se cambiarán las fechas mínimas y máximas en base a las fechas ingresadas en las facturas existentes)

```
Ingrese fecha final:
```

2. Análisis de datos:

Luego, dentro del método principal, se guarda el valor retornado por el método *ingresarFechas()* (las fechas a analizar) y se usa en el método *analisis()* de la funcionalidad **EvaluacionOperacion** (Definido en la línea 116).

Esta parte se puede observar en el código de la siguiente manera:

```
38 public static void evaluacionOperacion(){
39
40     int[] fechas = ingresarFechas();
41     analisis(fechas[0], fechas[1]);
42
43 }
```

A continuación, en el método *analisis()* se realizan todos los cálculos relacionados a la funcionalidad, como lo son las ganancias por día, las ganancias totales, el promedio por día, y el porcentaje de aumento.

Lo primero que se hace es encontrar las ganancias por día, que se guarda como una lista de parejas (clave, valor), donde la clave es la fecha y el valor es la ganancia en esa fecha específica.

Esta parte se puede observar en el código de la siguiente manera:

```
115 public static void analisis(int fechal, int fecha2){
116
117     if(fechal == -1 && fecha2 == -1){
118
119         return;
120     }
121
122     int fechaMin = Factura.getFechaMin();
123     int fechaMax = Factura.getFechaMax();
124
125     String opciones[] = new String[]{"Ganancias por día", "Ganancias Totales",
126         "Promedio por día", "Porcentaje de aumento", "Modas estadísticas", "Cambiar fechas"};
127
128     String opcion = new MenuAuxiliar("Ingrese información a obtener", opciones, "Volver al inicio").mostrarReturnString();
129
130     ArrayList<Parejas<Integer, Double>> disc = Factura.gananciasPorDia(fechal, fecha2);
131 }
```

Como podemos ver, para obtener esas ganancias por día se utiliza el método *gananciasPorDia()* de la clase **Factura** (Con sobrecarga, definido en las líneas 180 y 218), donde se recorren las fechas para sumar la ganancia correspondiente a cada factura.

Luego, en base a la información obtenida en *GananciasPorDia()* se calcula el resto de los valores mencionados anteriormente, así:

3. Ganancias Totales: Calculadas utilizando el método *ganancias()* de la clase **Factura** (Definido en la línea 235).
4. Promedio por Día: Calculado utilizando el método *promedioPorDia()* de la clase **Factura** (Definido en la línea 259).
5. Porcentaje de Aumento: Calculado utilizando el método *porcentajeDeAumento()* de la clase **Factura** (Definido en la línea 276).

Esta parte se puede observar en el código de la siguiente manera:

```

132         while(opcion!= "Volver al inicio"){
133
134             switch(opcion){
135
136                 case "GANANCIAS POR DIA":
137
138                     desplegarInfo(gananciasPorDia, "GANANCIA", "");
139
140                     break;
141
142                 case "GANANCIAS TOTALES":
143
144                     System.out.println("\nLas ganancias totales entre las fechas ingresadas han sido: " + Factura.ganancias(gananciasPorDia));
145
146                     break;
147
148                 case "PROMEDIO POR DIA":
149
150                     System.out.println("\nEl promedio por día es: " + Factura.promedioPorDia(gananciasPorDia));
151
152                     break;
153
154                 case "PORCENTAJE DE AUMENTO":
155
156                     desplegarInfo(Factura.porcentajeDeAumento(gananciasPorDia), "AUMENTO", "%");
157
158                     break;
159
160             }
161         }

```

También, hay un apartado para encontrar las modas estadísticas, que hace uso del método *modas()* de la funcionalidad **EvaluacionOperacion** (Definido en la línea 212) para hacer los cálculos correspondientes. El funcionamiento de esta parte se explicará más adelante.

Esta parte se puede observar en el código de la siguiente manera:

```

179         case "MODAS ESTADISTICAS":
180
181             modas(fecha1, fecha2);
182
183             break;

```

Por último, en esta sección se añade un apartado que permite hacer un cambio en las fechas ingresadas en caso de ser necesario, este se encuentra en el método *analisis()*.

Esta parte se puede observar en el código de la siguiente manera:

```

164         case "CAMBIAR FECHAS":
165
166             System.out.println("\nLa fecha mínima es " + fechaMin + " y la fecha máxima es " + fechaMax);
167
168             System.out.println("Ingrese fecha de inicio: ");
169             fecha1 = MenuAuxiliar.ingresarConLimites(fechaMin, fechaMax);
170
171             System.out.println("Ingrese fecha final: ");
172             fecha2 = MenuAuxiliar.ingresarConLimites(fecha1, fechaMax);
173
174             disc = Factura.gananciasPorDia(fecha1, fecha2);
175
176             break;
177     }

```

Esta parte se puede ver en la consola al ejecutar el código de la siguiente manera:

Ingrese información a obtener

1. Ganancias por día
2. Ganancias Totales
3. Promedio por día
4. Porcentaje de aumento
5. Modas estadísticas
6. Cambiar fechas
0. Cancelar

Los apartados de “Ganancias por día”, “Ganancias totales”, “Promedio por día”, “Porcentaje de aumento” y “Cambiar fechas” son los que han sido descritos en esta sección, estos se calculan de manera automática al seleccionar la opción y muestra el resultado en la pantalla.

3. Despliegue de información:

Como podemos ver, la información calculada se despliega utilizando el método *desplegarInfo()* de la funcionalidad **EvaluacionOperacion** (Definido en la línea 279), que muestra los datos en formato de tabla.

Esta parte se puede observar en el código de la siguiente manera:

```
279 public static void desplegarInfo(ArrayList<Parejas<Integer, Double>> info, String strData, String posfijo){
280
281     System.out.println("DIA \t " + strData);
282
283     for(Parejas<Integer, Double> par : info){
284
285         System.out.println(par.getKey() + "\t" + par.getValue() + posfijo);
286     }
287 }
```

4. Modas estadísticas:

El método *modas()* de la funcionalidad **EvaluacionOperacion** (Definido en la línea 212) se utiliza para encontrar las tiendas, clientes y transportes más usados. El llamado a este método se hace desde el método *analisis()* como se mostro anteriormente.

Esta parte se puede observar en el código de la siguiente manera:

```

212 public static void modas(int fechal, int fecha2){
213
214     String opciones[] = new String[]{"Tienda más usada", "Transporte más usado", "Cliente al que más se le ha vendido"};
215
216     int opcion = new MenuAuxiliar("Seleccione moda: ", opciones).mostrar();
217
218     System.out.println("\n");
219
220     while(opcion != 0){
221
222         // Manejo de opciones
223         switch(opcion){
224
225             case 1:
226
227                 // Uso de ligadura dinamica
228                 System.out.println("La tienda más usada ha sido " + Factura.moda(fechal, fecha2, "tienda").getNombre());
229
230                 break;
231
232
233             case 2:
234
235                 // Uso de ligadura dinamica
236                 System.out.println("El transporte más usado ha sido " + Factura.moda(fechal, fecha2, "transporte").getNombre());
237
238                 break;
239
240
241             case 3:
242
243                 // Uso de ligadura dinamica
244                 System.out.println("El cliente al que más se le ha vendido ha sido " + Factura.moda(fechal, fecha2, "cliente").getNombre());
245
246                 break;
247
248
249             case 0:
250
251                 return;
252
253             default:
254
255                 System.out.println("Ha ingresado un valor no válido. Por favor vuelva a intentarlo");
256
257                 break;
258         }
259
260         System.out.println("\n");
261
262         opcion = new MenuAuxiliar("Seleccione moda: ", opciones).mostrar();
263     }
264 }
265

```

Para el funcionamiento de este método se hace uso de la interfaz **Moda**, que se utiliza para marcar los objetos que pueden ser contados y a los que se les puede asignar una moda. Adicionalmente, los atributos se guardan en la lista *infoAtributos* de la clase **Factura**, que almacena parejas de valores (nombre del atributo, objeto que implementa **Moda**).

- Declaración del atributo:

```

45 private static ArrayList<Parejas<String, Moda>> infoAtributos = new ArrayList<>();

```

- Uso en el constructor de la clase **Factura**:

```

85         infoAtributos.add(new Parejas<>("tienda", tienda));
86         infoAtributos.add(new Parejas<>("transporte", transporte));
87         infoAtributos.add(new Parejas<>("cliente", cliente));

```

Por último, se utiliza el método *moda()* de la clase **Factura** (Definido en la línea 381), el cual retorna el resultado del método *masComun()* de la clase **Factura** (Definido en la línea 335) para encontrar el objeto más usado o la moda.

- Método *moda()*:

```

381 public static Moda moda(int inicio, int fin, String atributo) {
382
383     ArrayList<Factura> facturas = Factura.facturasPorPeriodo(inicio, fin);
384     ArrayList<Moda> objetos = new ArrayList<>();
385
386     for (Factura factura : facturas) {
387
388         objetos.add(obtenerValor(factura.getAtributos(), atributo));
389     }
390     return Factura.masComun(objetos);
391 }

```

- Método *masComun()*:

```

335 public static <T> T masComun(List<T> lista) {
336
337     ArrayList<Parejas<T, Integer>> map = new ArrayList<>();
338
339     for (T t : lista) {
340
341         boolean buscar = false;
342
343         for (Parejas<T, Integer> par : map) {
344
345             if (par.getKey().equals(t)) {
346                 par.setValue(par.getValue() + 1);
347                 buscar = true;
348                 break;
349             }
350         }
351         if (!buscar) {
352             map.add(new Parejas<>(t, 1));
353         }
354     }
355     Parejas<T, Integer> max = null;
356
357     for (Parejas<T, Integer> e : map) {
358
359         if (max == null || e.getValue() > max.getValue()) // Si cumple una o la otra
360             max = e;
361     }
362     return max.getKey();
363 }

```

Por último, es necesario aclarar que para el desarrollo de esta funcionalidad se hace una revisión de que cada uno de los datos ingresados por el usuario sean válidos para evitar errores y garantizar que los cálculos realizados sean correctos.

Manual de usuario

Para poder hacer uso de las funcionalidades en este proyecto, se vuelve necesario contar con ciertos objetos que permitan que se den las interacciones que se buscan, para esto se crearon los siguientes objetos de prueba:

- **Productos:** Se crearon 40 productos diferentes, los cuales están divididos de manera equitativa en los cuatro tipos de productos que se venden en el supermercado, estos son de tipo frutas y verduras, panadería, salsas y mermeladas, o bebidas. Estos fueron creados especificando atributos como el nombre, la descripción, el valor, el peso, el tamaño, el costo de producción y la categoría a la que pertenecen.
- **Cuentas bancarias:** Se creó una cuenta bancaria perteneciente a la empresa, la cual se encarga de recibir todos los pagos, gestionar los pagos a los empleados y realizar las devoluciones de dinero en caso de ser necesario. También, se creó una cuenta bancaria para cada empleado y cliente que será creado más adelante.
- **Tiendas:** Se crearon tres tiendas:
 - Delicia Fresca Norte: Con 25 productos variados entre las cuatro categorías.
 - Delicia Fresca Sur: Con 25 productos variados entre las cuatro categorías.
 - Delicia Fresca Centro: Con 30 productos variados entre las cuatro categorías (Cuenta con más productos por ser la sede central).
- **Fabrica:** Se creó una sola fábrica donde se fabrican o preparan cada uno de los 40 productos que han sido creados.
- **Operario:** Se creó un Operario que será asignado a la fábrica.
- **Vendedor:** Se crearon tres vendedores, los cuales serán asignados cada uno a una de las tres tiendas creadas
- **Transportador:** Se crearon tres transportadores.
- **Transporte:** Se crea un transporte.
- **Clientes:** Se crean ocho clientes diferentes, cada uno con sus propios atributos.

Interacción con las funcionalidades:

Después de iniciar la ejecución del programa podrá interactuar con las funcionalidades creadas. A continuación, se explicará cómo se pueden hacer estas interacciones.

Al iniciar la ejecución, lo primero que se verá es el menú principal del proyecto, aquí el usuario debe seleccionar una de las opciones disponibles para acceder a las funcionalidades, ingresando el número en la terminal correspondiente a la funcionalidad deseada. (Se escribe el número y luego se da enter para continuar).

Es importante aclarar que, en esta parte al igual que en todas las entradas del usuario, se realiza una verificación de que la opción ingresada exista para evitar errores.

```
Menú principal Delicia Fresca

Ingrese el número de la opción que desea utilizar
1. Envío pedidos
2. Pago de nomina
3. Proveer tiendas
4. Devoluciones de productos
5. Evaluación operación
6. Salir
```

- **Opción 1: Funcionalidad Envío pedidos**

Al seleccionar esta opción, se desplegará una lista con los clientes disponibles para realizar los envíos. Aquí el usuario debe seleccionar a que cliente desea hacer el envío, para esto debe ingresar el numero correspondiente y dar enter para continuar.

```
> 1

Seleccione el cliente al que desea enviar:
0. Volver al menu anterior

1. Esteban Carrillo Dirección: Calle 64 # 20
2. Tiffany Mendoza Dirección: Carrera 89 # 17
3. Santiago Perez Dirección: Avenida 35 # 21
4. Mariana Lopera Dirección: Carrera 23 # 56
5. Jose Manuel Vergara Dirección: Calle 65 # 21
6. Camila Zapata Dirección: Avenida 12 # 56
7. Miguel Acosta Dirección: Carrera 37 # 67
8. Martin Berrio Dirección: Calle 45 # 45
```

En este caso, seleccionaremos al primer cliente “Esteban Carrillo”. Para esto se debe escribir en la terminal el número 1 y luego dar enter para continuar.

```
> 1
Has seleccionado al cliente #1
El cliente es: Esteban Carrillo
```

Luego de seleccionar el cliente, se verá una lista con las tres tiendas y los productos que cada una tiene disponibles en el momento.

```
Su pedido se enviará desde alguna de estas tiendas, por favor seleccione una:
0. Volver al menu principal
1. Delicia Fresca Norte-Productos:
Manzana: 4
Zanahoria: 4
Pepino: 4
Limon: 4
Piña: 4
Pera: 4
Pan: 4
Sanduche: 4
Pandebono: 4
Pan hojaldrado: 4
Galletas: 4
Palito de queso: 4
Mermelada de frambuesa: 4
Mermelada de piña: 4
Mermelada de mora: 4
Mayonesa: 4
Salsa de tomate: 4
Mermelada de fresa: 4
Te verde: 4
Jugo de naranja: 4
Leche de almendras: 4
Leche de avena: 4
Yogur de soya: 4
Cafe instantaneo: 4
Jugo de mora: 4
```

2. Delicia Fresca Sur-Productos:

Pera: 4
Tomate: 4
Frambuesa: 4
Lechuga: 4
Sandia: 4
Pan: 4
Palito de queso: 4
Tostada: 4
Ponque: 4
Pan tajado: 4
Almojabana: 4
Mermelada de frambuesa: 4
Mermelada de fresa: 4
Mermelada de kiwi: 4
Mermelada de frutos rojos: 4
Salsa BBQ: 4
Salsa de soya: 4
Te verde: 4
Cafe instantaneo: 4
Bebida achocolatada: 4
Leche de coco: 4
Bebida hidratante: 4
Jugo de mora: 4
Pepino: 4
Jugo de naranja: 4

```
3. Delicia Fresca Centro-Productos:
Manzana: 4
Zanahoria: 4
Limon: 4
Piña: 4
Tomate: 4
Frambuesa: 4
Lechuga: 4
Sandia: 4
Sanduche: 4
Pandebono: 4
Pan hojaldrado: 4
Galletas: 4
Tostada: 4
Ponque: 4
Pan tajado: 4
Almojabana: 4
Mermelada de piña: 4
Mermelada de mora: 4
Mayonesa: 4
Salsa de tomate: 4
Mermelada de kiwi: 4
Mermelada de frutos rojos: 4
Salsa BBQ: 4
Salsa de soya: 4
Leche de almendras: 4
Leche de avena: 4
Yogur de soya: 4
Bebida achocolatada: 4
Leche de coco: 4
Bebida hidratante: 4

Seleccione la tienda desde la que desea enviar:
>
```

En este caso, seleccionaremos la primera tienda “Delicia Fresca Norte”. Para esto se debe escribir en la terminal el número 1 y luego dar enter para continuar.

```
Seleccione la tienda desde la que desea enviar:
> 1
Has seleccionado la tienda: 1
```

Luego de seleccionar la tienda, se pregunta por el número de productos que van a ser enviados al cliente. Se debe tener en cuenta que el número máximo de productor por envío son 5, por lo que el número seleccionado debe estar entre 1 y 5.

En este caso, se enviarán dos productos. Para esto, se debe escribir en la terminal el número 2 y luego dar enter para continuar.

```
¿Cuántos productos deseas comprar de esta tienda?  
Máximo 5 productos por cliente  
2
```

Luego de seleccionar el número de productos, se deben seleccionar los productos a enviar. Aquí se mostrar una lista con los productos disponibles, y se debe ingresar uno a uno el número correspondiente a los productos que se desean enviar. Es decir, se ingresa el número del primer producto que se desee enviar, se da enter, y luego el programa actualiza la cantidad disponible de cada producto para que se pueda ingresar el número del siguiente producto, y así se repite según la cantidad de productos que haya sido ingresada anteriormente.

En este caso, como seleccionamos que la cantidad de productos es dos, debemos seleccionar dos productos. Para el primer producto, la lista se ve así:

```
Seleccione el producto que desea enviarle al cliente  
0. Regresar al menu principal  
  
1. Manzana: 4  
2. Zanahoria: 4  
3. Pepino: 4  
4. Limon: 4  
5. Piña: 4  
6. Pera: 4  
7. Pan: 4  
8. Sanduche: 4  
9. Pandebono: 4  
10. Pan hojaldrado: 4  
11. Galletas: 4  
12. Palito de queso: 4  
13. Mermelada de frambuesa: 4  
14. Mermelada de piña: 4  
15. Mermelada de mora: 4  
16. Mayonesa: 4  
17. Salsa de tomate: 4  
18. Mermelada de fresa: 4  
19. Te verde: 4  
20. Jugo de naranja: 4  
21. Leche de almendras: 4  
22. Leche de avena: 4  
23. Yogur de soya: 4  
24. Cafe instantaneo: 4  
25. Jugo de mora: 4
```

Como primer producto, seleccionaremos “Manzana” (Producto #1). Para esto, se debe escribir en la terminal el número 1 y luego dar enter para continuar.

```
> 1
```

```
Ha seleccionado el producto # 1 Nombre del producto: Manzana
```

Luego, se actualiza la lista, cambiando las cantidades y número o índice de los productos. En este caso, se ve así:

```
Seleccione el producto que desea enviarle al cliente  
0. Regresar al menu principal
```

```
1. Zanahoria: 4  
2. Pepino: 4  
3. Limon: 4  
4. Piña: 4  
5. Pera: 4  
6. Pan: 4  
7. Sanduche: 4  
8. Pandebono: 4  
9. Pan hojaldrado: 4  
10. Galletas: 4  
11. Palito de queso: 4  
12. Mermelada de frambuesa: 4  
13. Mermelada de piña: 4  
14. Mermelada de mora: 4  
15. Mayonesa: 4  
16. Salsa de tomate: 4  
17. Mermelada de fresa: 4  
18. Te verde: 4  
19. Jugo de naranja: 4  
20. Leche de almendras: 4  
21. Leche de avena: 4  
22. Yogur de soya: 4  
23. Cafe instantaneo: 4  
24. Jugo de mora: 4  
25. Manzana: 3
```

Como segundo producto, seleccionaremos “Te verde” (Producto #18). Para esto, se debe escribir en la terminal el número 18 y luego dar enter para continuar.

```
> 18
```

```
Ha seleccionado el producto # 18 Nombre del producto: Te verde
```

Luego de seleccionar los productos, se debe seleccionar el medio de transporte en que se enviará el pedido. Aquí, se mostrará una lista que contiene los tipos de transporte que soportan el peso del pedido, se debe seleccionar alguno de estos.

```
Seleccione en que medio de transporte quiere enviar este producto
Junto a cada tipo de transporte se encuentra su precio.

Advertencia: Los tipos de transporte han sido filtrados de manera que solo puede seleccionar los que puedan soportar el peso de su producto.
Su pedido pesa 2 kilogramos
0. Regresar al menu principal
1. Caminando $1000.0
2. Bicicleta $2000.0
3. Moto $5000.0
4. Carro $10000.0
5. Camion $50000.0
6. Avion $500000.0

Seleccione el número del tipo de transporte que desea:
>
```

En este caso, se enviará el pedido en “Camión”. Para esto, se debe escribir en la terminal el número 5 y luego dar enter para continuar.

```
> 5
Ha seleccionado el transporte #5
El pedido se enviará por Camion
```

Luego de seleccionar el medio de transporte, se preguntará si quiere que se le aplique envío gratis al pedido.

```
Desea aplicar envío gratis?
1. Si    2. No
```

En este caso, si queremos aplicar envío gratis. Para esto, se debe escribir en la terminal el número 1 y luego dar enter para continuar. Sin embargo, si no se quisiera aplicar envío gratis, el programa continuaría sin problema

```
> 1
Su tarifa de envío ha bajado de: $50000.0 a $0
```

Luego, se debe ingresar el día del mes en que se esta haciendo el pedido. En este caso, diremos que el pedido se hizo el primer día del mes. Para esto, se debe escribir en la terminal el número 1 y luego dar enter para continuar.

```
Digite el día del mes entre 1 y 30:  
> 1
```

Aquí ya estaría completado el envío, por lo que se despliega la información de la factura en la pantalla.

```
-----  
Factura generada en la tienda Delicia Fresca Norte  
A nombre del cliente: Esteban Carrillo  
Nombre del producto: Manzana - Precio del producto: 8000.0  
Nombre del producto: Te verde - Precio del producto: 3000.0  
Tipo de transporte: CAMION  
Tarifa de envio: 0.0  
Total a pagar: 11000.0  
-----
```

Por último, se pregunta si se quiere volver al menú principal o realizar otro envío.

```
¿Desea hacer otro envio o volver al menu principal?  
0. Volver al menu principal  
1. Realizar otro envio
```

En este caso, se recomienda que se realicen más envíos para poder apreciar mejor el funcionamiento de algunas de las funcionalidades. Para esto, se debe escribir en la terminal el número 1 y luego dar enter para continuar.

De la misma manera en que se realizó este envío, se recomienda realizar los siguientes, para poder apreciar mejor el funcionamiento de las otras funcionalidades:

- **Envío 2:**
Cliente: 3. Santiago Perez Dirección: Avenida 35 # 21
Tienda: 2. Delicia Fresca Sur
Cantidad de productos: 3

Productos: 10. Pan tajado, 20. Leche de coco, 9. Ponque

Tipo de transporte: 3. Moto

Envío gratis: 1. Si

Día: 2

```
-----  
Factura generada en la tienda Delicia Fresca Sur  
A nombre del cliente: Santiago Perez  
Nombre del producto: Pan tajado - Precio del producto: 8500.0  
Nombre del producto: Leche de coco - Precio del producto: 15000.0  
Nombre del producto: Ponque - Precio del producto: 6000.0  
Tipo de transporte:          MOTO  
Tarifa de envio:            0.0  
Total a pagar:              29500.0  
-----
```

- **Envío 3:**

Cliente: 4. Mariana Lopera Dirección: Carrera 23 # 56

Tienda: 3. Delicia Fresca Centro

Cantidad de productos: 1

Productos: 8. Sandia

Tipo de transporte: 1. Caminando

Envío gratis: 1. Si

Día: 3

```
-----  
Factura generada en la tienda Delicia Fresca Centro  
A nombre del cliente: Mariana Lopera  
Nombre del producto: Sandia - Precio del producto: 12000.0  
Tipo de transporte:          CAMINANDO  
Tarifa de envio:            0.0  
Total a pagar:              12000.0  
-----
```

- **Envío 4:**

Cliente: 8. Martin Berrio Dirección: Calle 45 # 45

Tienda: 2. Delicia Fresca Sur

Cantidad de productos: 2

Productos: 22. Jugo de naranja, 7. Palito de queso

Tipo de transporte: 3. Moto

Envío gratis: 1. Si

Día: 4

```
-----  
Factura generada en la tienda Delicia Fresca Sur  
A nombre del cliente: Martin Berrio  
Nombre del producto: Jugo de naranja - Precio del producto: 3000.0  
Nombre del producto: Palito de queso - Precio del producto: 2500.0  
Tipo de transporte:          MOTO  
Tarifa de envio:             0.0  
Total a pagar:               5500.0  
-----
```

- **Envío 5:**

Cliente: 1. Esteban Carrillo Dirección: Calle 64 # 20

Tienda: 2. Delicia Fresca Sur

Cantidad de productos: 1

Productos: 11. Mermelada de kiwi

Tipo de transporte: 2. Bicicleta

Envío gratis: 1. Si

Día: 5

```
-----  
Factura generada en la tienda Delicia Fresca Sur  
A nombre del cliente: Esteban Carrillo  
Nombre del producto: Mermelada de kiwi - Precio del producto: 15000.0  
Tipo de transporte:          BICICLETA  
Tarifa de envio:             0.0  
Total a pagar:               15000.0  
-----
```

Luego de realizar estos envíos adicionales seleccionaremos 0 para volver al menú principal.

```
¿Desea hacer otro envio o volver al menu principal?  
0. Volver al menu principal  
1. Realizar otro envio  
> 0
```

- **Opción 2: Funcionalidad Pago de nomina**

Al seleccionar esta opción, se mostrarán los tipos de empleados existentes y el usuario debe seleccionar a que tipo de empleado desea hacer el pago, para esto debe ingresar el numero correspondiente y dar enter para continuar.

```
> 2

Has escogido la opción pagar a trabajadores

¿A qué tipo de trabajador desea pagarle?

1. Operarios
2. Transportadores
3. Vendedores
0. Volver al menú principal
```

En este caso, se le pagará a un Operario. Para esto, se debe escribir en la terminal el número 1 y luego dar enter para continuar.

Al hacerlo, se desplegará un mensaje con la información referente a los trabajadores del tipo seleccionado (En este caso, operarios) que tengan pagos pendientes.

```
> 1

Esta es la lista de Operarios que han trabajado

Trabajador 1
Nombre:      Mateo Lopez
Edad:        22
Identificación: 11111111
Fabrica:      Fábrica Delicia Fresca

Por favor, ingrese el número del trabajador al cual desea pagarle [1]:
```

En este caso, se le pagará al operario “Mateo Lopez”. Para esto, se debe escribir en la terminal el número 1 y luego dar enter para continuar.

Luego, se mostrará la información referente a las veces que ha trabajado y el pago que le corresponder por ese trabajo. Adicionalmente, se pregunta si se desea bonificar al trabajador teniendo en cuenta sus metas.

```
> 1

Al trabajador Mateo Lopez se le pagará 325000 por trabajar 5 veces

¿Desea analizar y bonificar al trabajador por sus metas cumplidas?
1.Si
2.No

Digite el número de la opción que desee
```

En este caso, queremos bonificar al empleado. Para esto, se debe escribir en la terminal el número 1 y luego dar enter para continuar.

```
> 1

El trabajador escogido tiene las siguientes metas

Indice: número de productos vendidos

Meta 1

Minimo requerido:          4.0
Bonificación por cumplimiento: 5000.0

Meta 2

Minimo requerido:          9.0
Bonificación por cumplimiento: 15000.0

Seleccione una de las metas
```

Al hacer esto, se desplegará la información de las metas del empleado escogido. Se debe elegir en base a lo que ha trabajado el empleado a cuál las metas se le verificará el cumplimiento.

En este caso, seleccionaremos la meta 1. Para esto, se debe escribir en la terminal el número 1 y luego dar enter para continuar.

```
Seleccione una de las metas
> 1

La meta ha sido cumplida exitosamente
Sumaremos al pago la bonificación por esta meta
Porcentaje de cumplimiento: 225.0%

¿Qué desea hacer?
  1.Revisar otra meta
  2.Proceder con el pago
```

Luego de ver que cumplió esta meta, revisaremos la siguiente meta. Para esto, se debe escribir en la terminal el número 1 y luego dar enter para continuar. Tras hacer esto, nos pedirá seleccionar la meta a verificar y elegiremos la meta 2. Para esto, se debe escribir en la terminal el número 2 y luego dar enter para continuar.

```
Seleccione una de las metas
> 2

La meta ha sido cumplida exitosamente
Sumaremos al pago la bonificación por esta meta
Porcentaje de cumplimiento: 100.0%

¿Qué desea hacer?
  1.Revisar otra meta
  2.Proceder con el pago
```

Luego de haber comprobado ambas metas, podremos proceder con el pago. Para esto, se debe escribir en la terminal el número 2 y luego dar enter para continuar.

Aquí también es importante tener en cuenta que si se quiere revisar la misma meta dos veces, el programa indicará que la meta ya ha sido revisada, y dará las mismas opciones de revisar otra meta o proceder con el pago.

```
> 2
... Realizando pago ... Por favor espere ...

El pago fue realizado con éxito

-----
Comprobante de pago
Pago asociado a los envios realizados 325000
Pago asociado al cumplimiento de metas 20000
Total = 345000
-----
```

Aquí, el pago a este trabajador queda realizado y nos muestra toda la información referente al pago.

Por último, se pregunta si se quiere volver al menú principal o pagar a otro trabajador. En este caso, volveremos al menú principal. Para esto, se debe escribir en la terminal el número 0 y luego dar enter para continuar.

```
¿Qué desea hacer?
1.Pagar a otro trabajador
0.Volver al menu principal
> 0
```

- **Opción 3: Funcionalidad Proveer tiendas**

Al seleccionar esta opción, se mostrarán las tiendas con los productos que tienen disponibles y la cantidad que hay de estos. El usuario debe seleccionar a qué tienda desea suministrar, para esto debe ingresar el numero correspondiente y dar enter para continuar.

> 3

Proveer tiendas - Apartado de tiendas

0. Volver al menu anterior

1. Delicia Fresca Norte-Productos:

Zanahoria: 4

Pepino: 4

Limon: 4

Piña: 4

Pera: 4

Pan: 4

Sanduche: 4

Pandebono: 4

Pan hojaldrado: 4

Galletas: 4

Palito de queso: 4

Mermelada de frambuesa: 4

Mermelada de piña: 4

Mermelada de mora: 4

Mayonesa: 4

Salsa de tomate: 4

Mermelada de fresa: 4

Jugo de naranja: 4

Leche de almendras: 4

Leche de avena: 4

Yogur de soya: 4

Cafe instantaneo: 4

Jugo de mora: 4

Manzana: 3

Te verde: 3

2. Delicia Fresca Sur-Productos:

Pera: 4
Tomate: 4
Frambuesa: 4
Lechuga: 4
Sandia: 4
Pan: 4
Tostada: 4
Almojabana: 4
Mermelada de frambuesa: 4
Mermelada de fresa: 4
Mermelada de frutos rojos: 4
Salsa BBQ: 4
Salsa de soya: 4
Te verde: 4
Cafe instantaneo: 4
Bebida achocolatada: 4
Bebida hidratante: 4
Jugo de mora: 4
Pepino: 4
Palito de queso: 3
Ponque: 3
Pan tajado: 3
Mermelada de kiwi: 3
Leche de coco: 3
Jugo de naranja: 3

3. Delicia Fresca Centro-Productos:

Manzana: 4
Zanahoria: 4
Limon: 4
Piña: 4
Tomate: 4
Frambuesa: 4
Lechuga: 4
Sanduche: 4
Pandebono: 4
Pan hojaldrado: 4
Galletas: 4
Tostada: 4
Ponque: 4
Pan tajado: 4
Almojabana: 4
Mermelada de piña: 4
Mermelada de mora: 4
Mayonesa: 4
Salsa de tomate: 4
Mermelada de kiwi: 4
Mermelada de frutos rojos: 4
Salsa BBQ: 4
Salsa de soya: 4
Leche de almendras: 4
Leche de avena: 4
Yogur de soya: 4
Bebida achocolatada: 4
Leche de coco: 4
Bebida hidratante: 4
Sandia: 3

CC

Seleccione la tienda a la que desea enviar:

En este caso, seleccionaremos la segunda tienda “Delicia Fresca Sur”. Para esto se debe escribir en la terminal el número 2 y luego dar enter para continuar.

Luego de seleccionar la tienda, se mostrará la capacidad de productos que se tiene para categoría en esta tienda. Además, se mostrarán los productos disponibles en la fabrica para abastecer con sus respectivas características (Nombre, peso, precio y categoría). Aquí se deberá elegir que producto se quiere suministrar, teniendo en cuenta las restricciones por categoría.

Seleccione la tienda a la que desea enviar: 2

Proveer tiendas - Apartado de productos

La capacidad de productos por categoria para esta tienda es la siguiente:

frutas y verduras 24/58 panaderia 21/84 salsas y mermeladas 23/81 bebidas 26/57

0. Regresar al menu anterior

NÚMERO-PRODUCTO-PESO-PRECIO-CATEGORIA

1. Manzana - 2.5 - 3000.0 - frutas y verduras
2. Zanahoria - 2.0 - 2000.0 - frutas y verduras
3. Pepino - 0.5 - 500.0 - frutas y verduras
4. Limon - 1.0 - 800.0 - frutas y verduras
5. Piña - 3.0 - 3500.0 - frutas y verduras
6. Pera - 2.5 - 4000.0 - frutas y verduras
7. Tomate - 2.0 - 1500.0 - frutas y verduras
8. Frambuesa - 1.0 - 6000.0 - frutas y verduras
9. Lechuga - 1.5 - 500.0 - frutas y verduras
10. Sandia - 5.0 - 4000.0 - frutas y verduras
11. Pan - 1.5 - 1500.0 - panaderia
12. Sanduche - 1.5 - 2500.0 - panaderia
13. Pandebono - 0.5 - 500.0 - panaderia
14. Pan hojaldrado - 0.5 - 800.0 - panaderia
15. Galletas - 1.5 - 400.0 - panaderia
16. Palito de queso - 0.5 - 500.0 - panaderia
17. Tostada - 1.0 - 3500.0 - panaderia
18. Ponque - 1.5 - 1500.0 - panaderia
19. Pan tajado - 1.5 - 2000.0 - panaderia
20. Almojabana - 0.5 - 1500.0 - panaderia
21. Mermelada de frambuesa - 1.5 - 3500.0 - salsas y mermeladas
22. Mermelada de piña - 1.2 - 3000.0 - salsas y mermeladas
23. Mermelada de mora - 0.8 - 2000.0 - salsas y mermeladas
24. Mayonesa - 2.5 - 6000.0 - salsas y mermeladas
25. Salsa de tomate - 1.5 - 2000.0 - salsas y mermeladas
26. Mermelada de fresa - 2.0 - 4500.0 - salsas y mermeladas
27. Mermelada de kiwi - 2.0 - 5000.0 - salsas y mermeladas
28. Mermelada de frutos rojos - 1.5 - 4000.0 - salsas y mermeladas
29. Salsa BBQ - 1.5 - 3000.0 - salsas y mermeladas
30. Salsa de soya - 1.0 - 1500.0 - salsas y mermeladas
31. Te verde - 0.5 - 300.0 - bebidas
32. Jugo de naranja - 2.0 - 1000.0 - bebidas
33. Leche de almendras - 1.5 - 4000.0 - bebidas
34. Leche de avena - 2.0 - 6000.0 - bebidas
35. Yogur de soya - 1.5 - 4500.0 - bebidas
36. Cafe instantaneo - 3.0 - 7000.0 - bebidas
37. Bebida achocolatada - 2.0 - 10000.0 - bebidas
38. Leche de coco - 1.5 - 6000.0 - bebidas
39. Bebida hidratante - 0.5 - 1500.0 - bebidas
40. Jugo de mora - 2.0 - 1500.0 - bebidas

Seleccione el producto que desea enviar:

En este caso, se enviará una mayonesa (Producto #24). Para esto, se debe escribir en la terminal el número 24 y luego dar enter para continuar.

```
Seleccione el producto que desea enviar: 24
Escriba la cantidad de productos que desea proveer:
```

Luego, se preguntará que cantidad de ese producto se desea enviar. En este caso, se enviarán 100. Para esto, se debe escribir en la terminal el número 100 y luego dar enter para continuar.

Al hacer esto, el sistema nos pedirá ingresar otro valor, ya que con esta cantidad se estaría pasando el máximo de productos de esa categoría que puede tener la tienda.

```
Por favor seleccione una cantidad en el limite de la tienda por categoria:
```

Ahora, intentaremos enviar 10 unidades. Para esto, se debe escribir en la terminal el número 10 y luego dar enter para continuar. Como esta cantidad entra dentro del rango permitido, el sistema la recibirá y avanzaremos al siguiente paso.

Luego de esto, se nos preguntará que medio de transporte queremos usar para realizar el envío. En este caso, se enviarán en el medio de transporte camión (Medio de transporte #2). Para esto, se debe escribir en la terminal el número 2 y luego dar enter para continuar.

```
Por favor seleccione una cantidad en el limite de la tienda por categoria: 10

Seleccione en que medio de transporte quiere enviar este producto

Advertencia: Los tipos de transporte han sido filtrados de manera que solo puede seleccionar los que puedan soportar el peso de su producto.
0. Regresar al menu anterior
1. Carro $10000.0
2. Camion $50000.0
3. Avion $500000.0

Seleccione el número del tipo de transporte:
```

Luego de seleccionar uno de los transportes, se proveerá a la tienda elegida con el producto y la cantidad elegida. Como confirmación se mostrarán las nuevas cantidades de cada producto en esta tienda.

```
> 2
Ha seleccionado el transporte #2
La tienda se proveerá por: Camion
El producto fue enviado con exito ahora la tienda tiene
PRODUCTOS:
Pera: 4
Tomate: 4
Frambuesa: 4
Lechuga: 4
Sandia: 4
Pan: 4
Tostada: 4
Almojabana: 4
Mermelada de frambuesa: 4
Mermelada de fresa: 4
Mermelada de frutos rojos: 4
Salsa BBQ: 4
Salsa de soya: 4
Te verde: 4
Cafe instantaneo: 4
Bebida achocolatada: 4
Bebida hidratante: 4
Jugo de mora: 4
Pepino: 4
Palito de queso: 3
Poncue: 3
Pan tajado: 3
Mermelada de kiwi: 3
Leche de coco: 3
Jugo de naranja: 3
Mayonesa: 10
```

Por último, se pregunta si se quiere volver al menú principal o proveer a otra tienda. En este caso, volveremos al menú principal. Para esto, se debe escribir en la terminal el número 0 y luego dar enter para continuar.

```
0.Volver al menu principal
1. Proveer más tiendas
> 0
```

- **Opción 4: Funcionalidad Devoluciones de productos**

Al seleccionar esta opción, se mostrarán las opciones de facturas de los envíos de pedidos que se tienen. En cada opción se muestra el nombre del cliente y la identificación de la compra, para poder elegir la factura deseada.

El usuario debe seleccionar a qué factura desea hacer la devolución, para esto debe ingresar el numero correspondiente y dar enter para continuar.

```
> 4

Por favor seleccione el número que le corresponda
a la factura para realizar la devoluciónn

0. Volver al menu principal
1. Cliente: Esteban Carrillo Identificación de compra1
2. Cliente: Santiago Perez Identificación de compra2
3. Cliente: Mariana Lopera Identificación de compra3
4. Cliente: Martin Berrio Identificación de compra4
5. Cliente: Esteban Carrillo Identificación de compra5

Digite su opcion:
-----
```

En este caso, se le hará la devolución a la primera factura. Para esto, se debe escribir en la terminal el número 1 y luego dar enter para continuar.

El número ingresado debe estar entre los índices que acompañan a cada factura. En este caso, deben estar entre 1 y 5. Adicionalmente, el sistema realiza una verificación de que todos los productos de esta factura no hayan sido ya devueltos. Si no se cumple alguna de estas condiciones pedirá ingresar otro número de factura.

Luego de verificar esto, se mostrarán los productos en esa factura y el usuario debe elegir el que desea devolver.

```
Seleccionó la factura con la opcion número: 1

Por favor seleccione el número que le corresponda
a el producto para realizar la devolución

0. Volver al menu principal
1. Producto: Manzana
2. Producto: Te verde

Digite su opcion:
```

En este caso, se le hará la devolución del “Té verde”. Para esto, se debe escribir en la terminal el número 2 y luego dar enter para continuar.

Aquí, se verificará que el número ingresado este dentro del rango y no haya sido devuelto antes, si cumple la condición, se realiza la devolución del producto, mostrando por consola el siguiente mensaje de confirmación.

```
> 2

Seleccionó el producto con la opcion número: 2
Se esta realizando devolución, por favor espere
El producto ha sido devuelto
```

Por último, se pregunta si se quiere volver al menú principal o realizar otra devolución. En este caso, volveremos al menú principal. Para esto, se debe escribir en la terminal el número 0 y luego dar enter para continuar.

```
¿Desea hacer otra devolucion o volver al menu principal?

0. Volver al menu principal
1. Realizar otra devolución
> 0
```

- **Opción 5: Funcionalidad Evaluación operación**

Al seleccionar esta opción, el usuario debe seleccionar si desea analizar toda la información o elegir unas fechas específicas, para hacer la selección debe ingresar el numero correspondiente y dar enter para continuar.

```
> 5
REPORTE

1. Analizar toda la información
2. Ingresar fechas específicas
0. Cancelar
```

En este caso, ingresaremos las fechas a analizar. Para esto, se debe escribir en la terminal el número 2 y luego dar enter para continuar.

```
> 2

La fecha mínima es 1 y la fecha máxima es 5
Ingrese fecha de inicio:
```

El programa indicará cual es el rango posible en las fechas en base a las facturas generadas en la funcionalidad de envío de pedidos, si se ingresa un valor de inicio o de final fuera de este rango nos pedirá ingresar otro valor.

Ahora, debemos elegir la fecha de inicio, que, en este caso, será el primer día del mes. Para esto, se debe escribir en la terminal el número 1 y luego dar enter para continuar.

```
> 1
Ingrese fecha final:
```

Ahora, debemos elegir la fecha final, que, en este caso, será el cuarto día del mes. Para esto, se debe escribir en la terminal el número 4 y luego dar enter para continuar.

```

> 4
Ingrese información a obtener

1. Ganancias por día
2. Ganancias Totales
3. Promedio por día
4. Porcentaje de aumento
5. Modas estadísticas
6. Cambiar fechas
0. Cancelar

```

Luego, sin importar si ingresamos las fechas o vamos a analizar toda la información, saldrá un nuevo menú con 6 opciones de análisis de la información. Aquí el usuario deberá elegir cual de estos datos desea consultar. A continuación, se mostrará cómo se ve cada una de las opciones al seleccionarlás (Ingresando el numero correspondiente):

1. Ganancias por día: Al seleccionar esta opción se mostrará las ganancias de cada día dentro de las fechas ingresadas o la información total, mostrando solo aquellos días en los que hubieron ganancias.

```

> 1
DIA      GANANCIA
1        11000.0
2        29500.0
3        12000.0
4         5500.0

```

Luego de mostrar esta información, se volverá a mostrar el menú para seleccionar alguna de las otras opciones.

```

Ingrese información a obtener

1. Ganancias por día
2. Ganancias Totales
3. Promedio por día
4. Porcentaje de aumento
5. Modas estadísticas
6. Cambiar fechas
0. Cancelar

```

2. Ganancias totales: Al seleccionar esta opción se mostrará las ganancias totales dentro de las fechas ingresadas o la información total.

> 2

Las ganancias totales entre las fechas ingresadas han sido: 58000.0

Luego de mostrar esta información, se volverá a mostrar el menú para seleccionar alguna de las otras opciones.

Ingrese información a obtener

1. Ganancias por día
2. Ganancias Totales
3. Promedio por día
4. Porcentaje de aumento
5. Modas estadísticas
6. Cambiar fechas
0. Cancelar

3. Promedio por día: Al seleccionar esta opción se mostrará el promedio por día dentro de las fechas ingresadas o la información total.

> 3

El promedio por día es: 14500.0

Luego de mostrar esta información, se volverá a mostrar el menú para seleccionar alguna de las otras opciones.

Ingrese información a obtener

1. Ganancias por día
2. Ganancias Totales
3. Promedio por día
4. Porcentaje de aumento
5. Modas estadísticas
6. Cambiar fechas
0. Cancelar

4. Porcentaje de aumento: Al seleccionar esta opción se mostrará el porcentaje de aumento en las ganancias respecto al día anterior.

> 4

DIA	AUMENTO
2	168.1818181818182%
3	-59.32203389830508%
4	-54.166666666666664%

Luego de mostrar esta información, se volverá a mostrar el menú para seleccionar alguna de las otras opciones.

```
Ingresa información a obtener

1. Ganancias por día
2. Ganancias Totales
3. Promedio por día
4. Porcentaje de aumento
5. Modas estadísticas
6. Cambiar fechas
0. Cancelar
```

- 5. Modas estadísticas: Al seleccionar esta opción se mostrarán las opciones de objetos a los que se les puede buscar una moda. Estas opciones son: Tienda, Transporte y Cliente.

```
> 5
Seleccione moda:

1. Tienda más usada
2. Transporte más usado
3. Cliente al que más se le ha vendido
0. Cancelar
```

- Si se selecciona la opción 1 (Tienda más usada), veremos la siguiente información:

```
> 1

La tienda más usada ha sido Delicia Fresca Norte
```

Luego de mostrar esta información, se volverá a mostrar el menú para seleccionar alguna de las otras opciones de modas.

```
Seleccione moda:

1. Tienda más usada
2. Transporte más usado
3. Cliente al que más se le ha vendido
0. Cancelar
```

- Si se selecciona la opción 2 (Transporte más usado), veremos la siguiente información:

```
> 2
El transporte más usado ha sido Camion
```

Luego de mostrar esta información, se volverá a mostrar el menú para seleccionar alguna de las otras opciones de modas.

```
Seleccione moda:

1. Tienda más usada
2. Transporte más usado
3. Cliente al que más se le ha vendido
0. Cancelar
```

- Si se selecciona la opción 3 (Cliente al que más se le ha vendido), veremos la siguiente información:

```
> 3
El cliente al que más se le ha vendido ha sido Esteban Carrillo
```

Luego de mostrar esta información, se volverá a mostrar el menú para seleccionar alguna de las otras opciones de modas.

```
Seleccione moda:

1. Tienda más usada
2. Transporte más usado
3. Cliente al que más se le ha vendido
0. Cancelar
```

- Si se selecciona la opción 0 (Cancelar), se volverá a mostrar el menú principal de la funcionalidad.

```
Ingrese información a obtener
```

```
1. Ganancias por día
2. Ganancias Totales
3. Promedio por día
4. Porcentaje de aumento
5. Modas estadísticas
6. Cambiar fechas
0. Cancelar
```

6. Cambiar fechas: Al seleccionar esta opción se mostrará de nuevo la instrucción para ingresar fechas

```
> 6
```

```
La fecha mínima es 1 y la fecha máxima es 5  
Ingrese fecha de inicio:
```

0. Cancelar: Al seleccionar esta opción, el programa volverá al menú principal

- **Opción 6:** Se termina la ejecución del programa

Al elegir la opción de salir, los cambios hechos se van a guardar automáticamente y se muestra el siguiente mensaje de salida:

```
> 6  
Gracias por visitarnos  
Vuelva pronto
```